

TEMA 3

INTRODUCCIÓN AL LENGUAJE VHDL



Tema 1. Análisis de los circuitos integrados digitales



Tema 2. Diseño de sistemas digitales mediante Dispositivos Lógicos Programables



Tema 3. Introducción al lenguaje VHDL



Tema 4. Statements concurrentes, tipos de datos numéricos y conversiones en el lenguaje VHDL



Tema 5. Procesos y atributos en VHDL



Tema 6. Máquinas de estados finitas (FSM) y su descripción en VHDL



Tema 7. Diseño jerárquico y parametrizable en VHDL



Tema 8. Diseño RTL en VHDL

Tema 3. Introducción al lenguaje VHDL

❖ Objetivos

- Definir los conceptos básicos de VHDL
 - ✓ Elementos básicos de modelación
 - ✓ Elemento léxicos
 - ✓ Descripciones sintácticas
- Analizar las partes mínimas que componen un fichero fuente VHDL
 - ✓ Entidad
 - ✓ Arquitectura
- Describir los conceptos necesarios para entender mejor los temas siguientes
 - ✓ Concurrencia
 - ✓ Jerarquía
- Describir las etapas que componen el flujo de diseño en FPGAs

Tema 3. Introducción al lenguaje VHDL

❖ Bibliografía

- John F. Wakerly. *Diseño Digital. Principios y Prácticas*. Ed. Prentice Hall. 2001.
- William Kafig. *VHDL 101. Everything you need to know to get started*. Ed. Newnes. 2011.
- Gina R. Smith. *FPGA 101. Everything you need to know to get started*. Ed. Newnes. 2010.
- Pong P. Chu. *FPGA Prototyping by VHDL Examples*. Ed. Wiley. 2008.
- Ion Grout. *Digital Systems Design with FPGAs and CPLDs*. Ed Elsevier Ltd. 2008.
- Peter Wilson. *Design Recipes for FPGAs*. Ed. Newnes. 2007.
- David Pellerin y otros. *VHDL Made Easy!*. Ed. Mc Graw Hill. 1997.
- P. J. Ashenden. *The Student's Guide to VHDL*. Ed. Morgan Kaufmann. 2008.
- Peter J. Ashenden. *The Designer's Guide to VHDL*. Ed. Elsevier Inc. 2008.
- Douglas Perry. *VHDL*. Ed. Mc-Graw Hill. 1998.
- R. C. Cofer and Benjamin F. Harding. *Rapid System Prototyping with FPGAs*. Ed. Elsevier Inc. 2006.

1. Introducción al Lenguaje VHDL. Un poco de historia

- **VHDL** corresponde a las siglas de **VHSIC Hardware Description Language**
 - ✓ **VHSIC** son las siglas de **Very High Speed Integrated Circuit**
- Fue desarrollado a principios de los 80 por el Departamento de Defensa de E.E.U.U.
 - ✓ Lenguaje para describir circuitos complejos que debían ser diseñados por varios grupos de trabajo
 - ✓ La especificación y los formatos descriptivos del lenguaje fueron desarrollados por un grupo de ingenieros de IBM, Texas Instruments e Intermetrics
- La primera versión pública de VHDL se publicó en 1985: versión 7.2
- En 1986 se presentó en el **IEEE** (Institute of Electrical and Electronics Engineers) para su estandarización
- En 1987 se publicó el estándar con la referencia **IEEE 1076-1987**
- En 1994 se publicó una versión mejorada y actualizada: **IEEE 1076-1993**
- Posteriormente se realizaron otras revisiones: **VHDL-2000, VHDL-2002 y VHDL-2008**

1. Introducción al Lenguaje VHDL. Aplicaciones

❖ Aplicaciones de VHDL:

- **Especificación de componentes.**
- **Modelado de componentes**
 - ✓ **VHDL** es un lenguaje de alto nivel que facilita la descripción del comportamiento de los componentes o incluso sistemas obteniendo un modelo simulable.
- **Descripción del diseño lógico de un componente o circuito**
 - ✓ Permite implementar físicamente un diseño mediante el uso de **herramientas de síntesis**.
 - ✓ No todas las descripciones son sintetizables.
 - ✓ Debe analizarse las recomendaciones indicadas en la **Guía de Usuario** de las herramientas.
 - ✓ Generalmente hay unos formatos admitidos por todas las herramientas.
- **Verificación del diseño**
 - ✓ Permite describir el **test bench** para generar los estímulos de la simulación, e incluso comprobar automáticamente el resultado de ésta.
- **Documentación del diseño.**

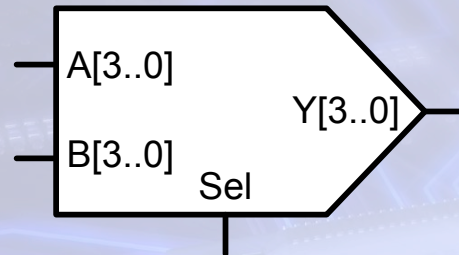


1. Introducción al Lenguaje VHDL. Ventajas

❖ Ventajas de VHDL:

- Ofrece muchas opciones de flexibilidad y configurabilidad en los diseños
- Es un lenguaje estándar
 - ✓ Compatibilidad con las herramientas de diseño de última y futura generación.
 - ✓ Los diseños son independientes de la tecnología:
 - Se describen **diseños portables**.
- Acceso a una amplia **base de conocimiento**.
 - ✓ Existen **cores gratuitos** que describen multitud de componentes:
 - UART, Microprocesadores, Microcontroladores, Controladores de bus, pila TCP/IP, etc.
 - Ejemplo: www.opencores.org

2. Elementos básicos de modelación



- La descripción de un modelo o diseño VHDL consta como mínimo de un **par entity/architecture**.
- Se denomina **entidad de diseño**.

Ejemplo de fichero fuente VHDL

Comentario → -- Mux de 2 a 1 de 4 bits

Declaración entidad

Declaración Puertos

Declaración arquitectura

Asignación condicional de señal

Entity mux2a1_4bits **is**

port(A, B : **in** bit_vector(3 **downto** 0); -- Descripción de puertos

Sel : **in** bit;

Y : **out** bit_vector(3 **downto** 0));

End mux2a1_4bits;

Architecture Funcional **of** mux2a1_4bits **is**

Begin

-- Descripción del comportamiento

Y <= A **when** Sel = '0' -- Y = A Si Sel es 0

else B -- de lo contrario (Sel = 1) Y = B

End Funcional;

2. Elementos básicos de modelación.

Elementos léxicos de VHDL

❖ Identificadores

Se usan para nombrar los elementos de un modelo VHDL

■ Normas

- ✓ Constan de **letras** ('A' ... 'Z' y 'a' ... 'z'), **dígitos decimales** ('0' al '9') y **el carácter underline** (_)
- ✓ VHDL **no distingue entre letras mayúsculas y minúsculas**:

Estos
identificadores
son idénticos

An_Example_Name
AN_EXAMPLE_NAME
an_example_name

- ✓ **Deben comenzar con una letra**
- ✓ **No pueden terminar con un carácter underline** (_)
- ✓ **No puede incluir dos caracteres underline seguidos**
- ✓ **Ejemplos de identificadores inválidos**

signal 16_BIT_BUS : integer; -- ilegal

signal _BUS_16_BIT : integer; -- ilegal

signal BUS_16_BIT : integer; -- legal

signal BUS_16_BIT_ : integer; -- ilegal

signal BUS_16__BIT : integer; -- ilegal

signal BUS_16@BIT : integer; -- ilegal

2. Elementos básicos de modelación.

Elementos léxicos de VHDL

Palabras clave (Keywords)

- Son identificadores reservados por VHDL, ya que tienen un significado especial
- Se usan para indicar construcciones específicas del lenguaje
- No se pueden usar como identificadores
- Se representan en negrita

abs	entity	next	ror	xor
access	exit	nor	select	
after	file	not	severity	
alias	for	null	signal	
all	function	of	shared	
and	generate	on	sla	
architecture	generic	open	sll	
array	group	or	sra	
assert	guarded	others	srl	
attribute	if	out	subtype	
begin	impure	package	then	
block	in	port	to	
body	inertial	postponed	transport	
buffer	inout	procedure	type	
bus	is	process	unaffected	
case	label	pure	units	
component	library	range	until	
configuration	linkage	record	use	
constant	literal	register	variable	
disconnect	loop	reject	wait	
downto	map	rem	when	
else	mod	report	while	
elsif	nand	return	with	
end	new	rol	xnor	

2. Elementos básicos de modelación. Elementos léxicos de VHDL

❖ VHDL es un lenguaje de formato libre

- No hay normas de formateo, tales como espaciamento o sangrado
- Los espacios y los retornos de carro (Carriage return) se consideran iguales
- Las dos descripciones siguientes se interpretan exactamente del mismo modo

```
-- Mux de 2 a 1 de 4 bits
Entity mux2a1_4bits is
  port(A, B : in bit_vector(3 downto 0);
    Sel : in bit;
    Y : out bit_vector(3 downto 0));
End mux2a1_4bits;
Architecture Funcional of mux2a1_4bits is
Begin
  --Descripción del comportamiento
  Y <= A when Sel = '0' -- Y = A Si Sel es 0
    else B;             -- de lo contrario (Sel = 1) Y = B
End Funcional;
```

```
-- Mux de 2 a 1 de 4 bits
Entity mux2a1_4bits is port(
  A, B : in bit_vector(3 downto 0);
  Sel : in bit;
  Y : out bit_vector(3 downto 0));
End mux2a1_4bits;
Architecture Funcional of mux2a1_4bits is
Begin
  --Descripción del comportamiento
  Y <= A when Sel = '0' else B;
    -- Y = A Si Sel es 0 de lo contrario (Sel = 1) Y = B
End Funcional;
```


2. Elementos básicos de modelación. Elementos léxicos de VHDL

❖ Comentarios

- Se usan para documentar el diseño
 - ✓ Explican ciertos detalles de la descripción para que ésta la comprenda más fácilmente otro diseñador, o el mismo autor pasado un tiempo.
- Sintaxis:
 - ✓ Se inician mediante un doble guión (--).
 - ✓ Se puede situar en cualquier parte de una línea:
 - Al principio o a continuación de un statement.
 - ✓ Todo lo que sigue a "--" será considerado como un comentario.
 - ✓ El retorno de carro (Carriage return) termina el comentario.
 - ✓ VHDL no permite comentarios de bloque.
 - Para continuar un comentario en otra línea debe iniciarse con "--".

```
-- Mux de 2 a 1 de 4 bits
Entity mux2a1_4bits is
  port(A, B : in bit_vector(3 downto 0);
        Sel : in bit;
        Y : out bit_vector(3 downto 0));
End mux2a1_4bits;
Architecture Funcional of mux2a1_4bits is
Begin
  -- Descripción del comportamiento
  -- Se usa un estamento condicional de señal
  Y <= A when Sel = '0'    -- Y = A Si Sel es 0
    else B;                -- de lo contrario (Sel = 1) Y = B
End Funcional;
```


2. Elementos básicos de modelación. Elementos léxicos de VHDL

❖ Descripciones sintácticas

- Se mostrarán las construcciones simplificadas representadas mediante el **formato modificado BNF (backus-Naur form)**

- ✓ **Palabras en negrita.** Representan palabras reservadas
- ✓ **Barra vertical (|).** Representa una elección entre elementos
- ✓ **Corchetes ([]).** Contiene elementos opcionales
- ✓ **Llaves ({ }).** Contiene elementos opcionales que pueden repetirse, excepto los elementos en negrita siguientes a la llave de apertura que debe incluirse

- **Ejemplo**

[signal] identificador {, identificador} : [mode] tipo_señal

3. Estructura de un diseño básico en VHDL.

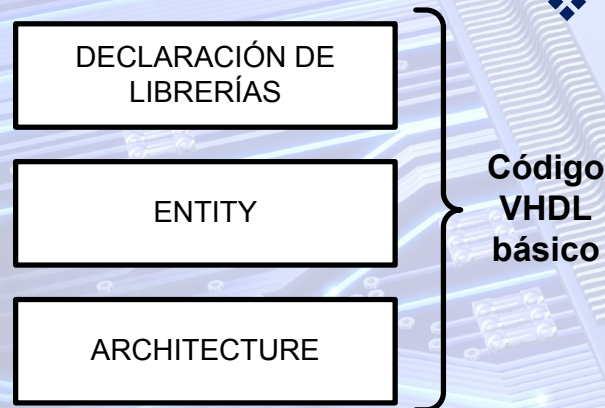
Unidades de diseño

❖ Concepto de unidad de diseño

- Segmentos de código VHDL que pueden compilarse independientemente y almacenarse en una librería
- **Tipos:**
 - ✓ **Entity (Entidad)**
 - ✓ **Architecture (Arquitectura)**
 - ✓ **Package (paquete) y package body (cuerpo del paquete)**
 - ✓ **Configuration:** concepto avanzado en VHDL que permite especificar diferentes arquitecturas para una misma entidad

❖ Estructura básica de una descripción VHDL

- Un fichero fuente VHDL básico suele constar por lo menos de tres secciones fundamentales:
 - ✓ **Declaración de librerías**
Indica las librerías que se usan en el diseño
Por ejemplo: ieee, std, work, etc.
 - ✓ **Entity**
Especifica las conexiones de entrada y salida del circuito
 - ✓ **Architecture**
Describe el comportamiento o la estructura del circuito



3. Estructura de un diseño básico en VHDL. Library (librería)

❖ Concepto de librería

- Colección de segmentos de código VHDL compilados
- Permite reusar o compartir ese código por otros diseños

▪ Contenido:

✓ Package y/o package body

- Describen Functions (funciones), procedures (procedimientos) o components (componentes) y tipos de datos.
- El package se compila en una librería.

▪ Packages estandarizados

- ✓ Las revisiones del estándar del 87 se hace mediante packages que se incluyen en la librería ieee.
- ✓ Estos packages generalmente definen nuevos tipos de datos y sus operadores.

▪ Ejemplos:

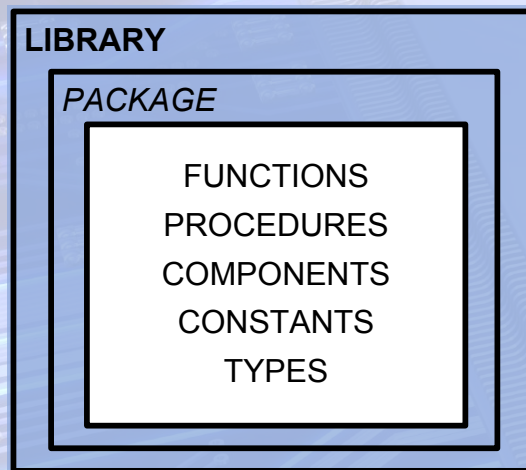
✓ Std_logic_1164

Declara el tipo std_logic y el std_logic_vector y sus operadores básicos: lógicos, concatenación, relacionales, etc.

✓ Std_logic_unsigned

Declara los operadores de suma, resta y multiplicación para el tipo std_logic_vector

Estructura de una librería



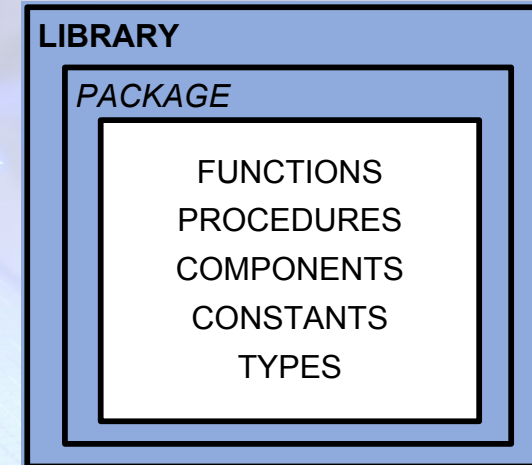
❖ Declaración de librerías

- Ejemplos

✓ **package standard** contenido en la **librería std**

✓ Cualquier compilación realizada en la **librería work**

Estructura de una librería



- Se suelen usar tres librerías: std, work e ieee.
- La librería STD es una librería de recursos con 2 paquetes (standar, textio)
 - Tipos básicos de datos del estándar del 87 para la descripción de circuitos (bit, integer, carácter...)
 - Entrada/salida de texto, etc.
- La librería WORK es donde se almacena los resultados de compilar todas las unidades de nuestro diseño (entidades, arquitecturas, paquetes, configuraciones) más todos los ficheros creados por el compilador y el simulador.
- Las librerías std y work son visibles por defecto.
 - Por tanto, no se tienen que declarar.

3. Estructura de un diseño básico en VHDL.

Entity (Entidad)

❖ Concepto de Entidad

- Define el **interfaz externo** de un circuito o componente y le asigna un **nombre** (identificador)
- Indica las conexiones externas.

❖ Sintaxis

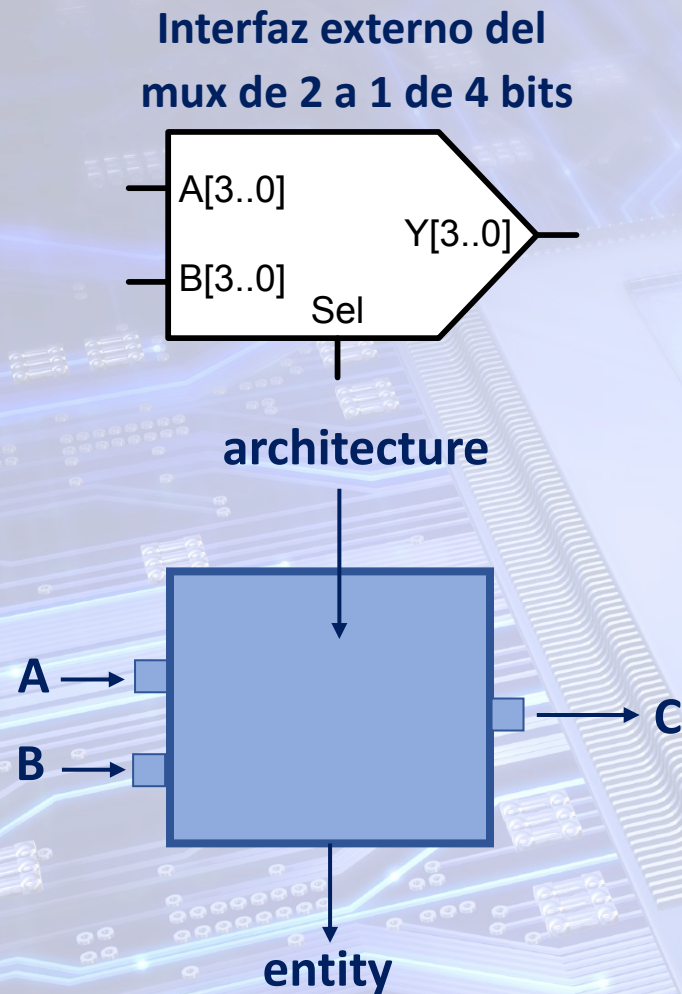
entity nom_entity **is**

[**generic** (lista de generic);]

port (identificador {, identificador} : [mode] tipo_dato
{; identificador {, identificador} : [mode] tipo_dato});

end [**entity**] nom_entity;

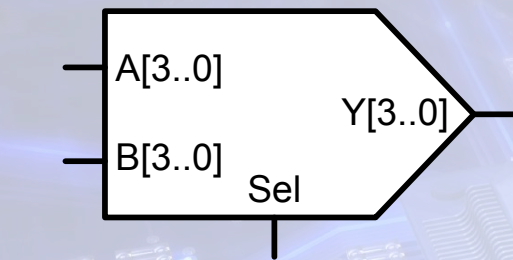
- Conviene usar el mismo nombre para la entidad que para el fichero fuente.
- Se deben listar mediante la cláusula **port** todas las conexiones (puertos) de entrada, salida y/o bidireccionales del circuito o del componente.
- Los nombres de la entidad y de los puertos son *identificadores*, por lo que siguen sus mismas reglas.
- Conviene usar nombres no muy largos (32 caracteres como máximo), que permitan identificar el componente y los puertos.



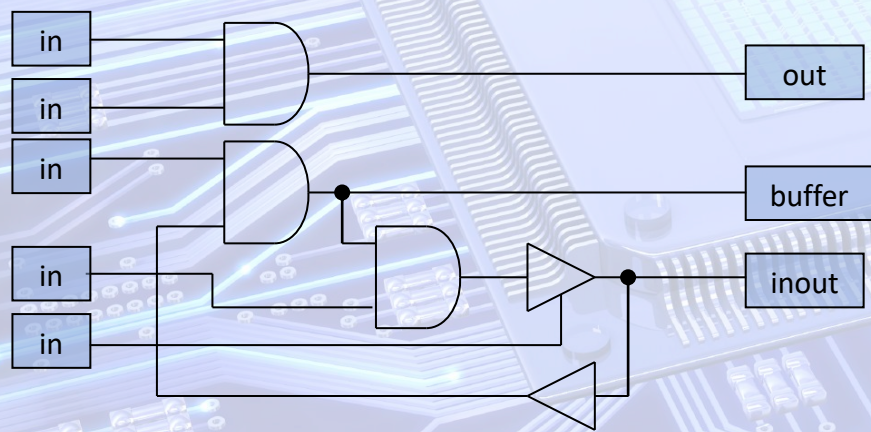
3. Estructura de un diseño básico en VHDL. Entity (Entidad)

❖ Declaración de los puertos

- Para cada **port** se debe indicar:
 - ✓ *Nombre identificador*
 - ✓ *Modo* = dirección: **in** | **out** | **inout** | **buffer** **Por defecto se considera in**
 - ✓ *Tipo de dato*. Indica el tipo de información que almacena y los operadores permitidos
- El modo determina si en la arquitectura se puede leer y/o escribirse.



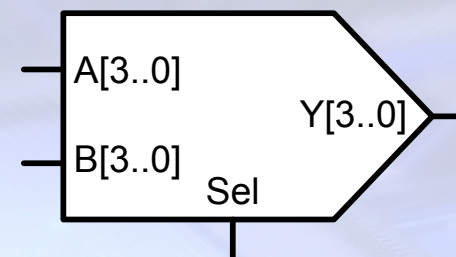
```
entity nom_entity is
  [generic (lista de generic);]
  port (identificador {, identificador} : [mode] tipo_dato
        {; identificador {, identificador} : [mode] tipo_dato});
end [entity] nom_entity;
```



Modo	Lectura	Escritura	Tipo
in	SI	NO	Entrada
out	NO	SI	Salida
buffer	SI	SI	Salida con realimentación
inout	SI	SI	Bidireccional

3. Estructura de un diseño básico en VHDL. Entity (Entidad)

```
entity nom_entity is
  [generic (lista de generic);]
  port (identificador {, identificador} : [mode] tipo_dato
        {; identificador {, identificador} : [mode] tipo_dato});
end [entity] nom_entity;
```



❖ Declaración de la entidad del multiplexor de 2 a 1 de 4 bits

-- Mux de 2 a 1 de 4 bits

Entity mux2a1_4bits **is port**(

A : **in** bit_vector(3 **downto** 0);

B : **in** bit_vector(3 **downto** 0);

Sel : **in** bit;

Y : **out** bit_vector(3 **downto** 0)

);

End mux2a1_4bits;

-- El nombre indica la función del diseño

-- Entrada de datos A. Array de 4 bits

-- Entrada de datos B. Array de 4 bits

-- Entrada de selección. Cada port termina con “;”

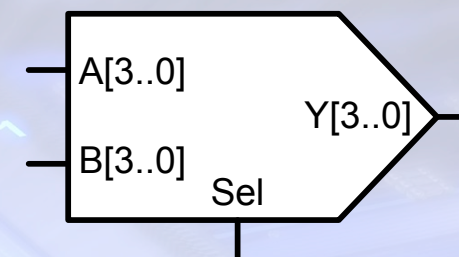
-- Salida de datos Y. Array de 4 bits

-- La declaración de los ports termina con “;”

-- Fin de la entidad

3. Estructura de un diseño básico en VHDL. Entity (Entidad)

```
entity nom_entity is
  [generic (lista de generic);]
  port (identificador {, identificador} : [mode] tipo_dato
        {; identificador {, identificador} : [mode] tipo_dato});
end [entity] nom_entity;
```



❖ Otra declaración alternativa de la entidad del multiplexor

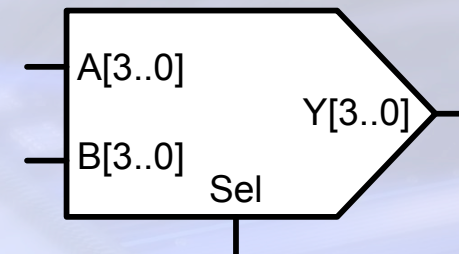
-- Mux de 2 a 1 de 4 bits

```
Entity mux2a1_4bits is port(
  A, B : bit_vector(3 downto 0);
  Sel : bit;
  Y : out bit_vector(3 downto 0)
);
End mux2a1_4bits;
```

-- El nombre indica la función del diseño
 -- Entrada de datos A y B. Array de 4 bits
 -- Entrada de selección. El modo por defecto es in
 -- Salida de datos Y. Array de 4 bits
 -- La declaración de los ports termina con ";"
 -- Fin de la entidad

3. Estructura de un diseño básico en VHDL. Entity (Entidad)

```
entity nom_entity is  
  [generic (lista de generic);]  
  port (identificador {, identificador} : [mode] tipo_dato  
        {; identificador {, identificador} : [mode] tipo_dato});  
end [entity] nom_entity;
```



❖ Generic

- Se usan para pasar parámetros adicionales a una entidad.
- Son opcionales.
- No tienen modo. Sólo tipo de datos.
- Sintaxis

```
generic (identificador {, identificador} : tipo_dato [:= valor_defec]);
```

▪ Utilidad

- ✓ Definir circuitos o componentes genéricos.
- ✓ Se particularizarán mediante la información que se le pase a la entidad en los generic.

▪ Ejemplo

- ✓ Mux de 2 a 1 de n bits

3. Estructura de un diseño básico en VHDL. Entity (Entidad)

generic (identificador {, identificador} : tipo_dato [:= valor_defec]);

❖ Declaración de la entidad del multiplexor de n bits con generic

-- Mux de 2 a 1 de n bits

Entity mux2a1_nbits **is**

generic (Num_bits : integer := 16);

-- Por defecto mux de 16 bits

port(

A : **in** bit_vector(Num_bits-1 **downto** 0); -- Entrada de datos A. Array de n bits

B : **in** bit_vector(Num_bits-1 **downto** 0); -- Entrada de datos B. Por defecto array de 16 bits

Sel : bit; -- Entrada de selección. El modo por defecto es **in**

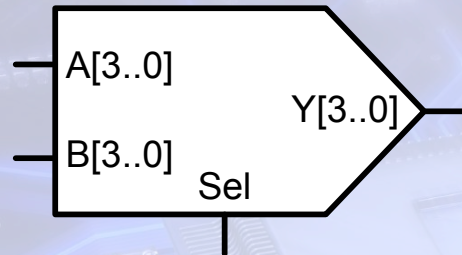
Y : **out** bit_vector(Num_bits-1 **downto** 0) -- Salida de datos Y. Array de n bits

);

End mux2a1_4bits;



3. Estructura de un diseño básico en VHDL. Architecture (Arquitectura)



❖ Concepto de Arquitectura

- Describe la función y/o la estructura de la entidad (circuito o componente).
- Cada architecture está asociada por un nombre a una entity.
- VHDL permite crear más de una architecture para cada entity.

❖ Sintaxis

```
architecture nom_architecture of nom_entity is  
    {declaración_elementos}  
begin  
    statements_concurrentes;  
end [nom_architecture];
```

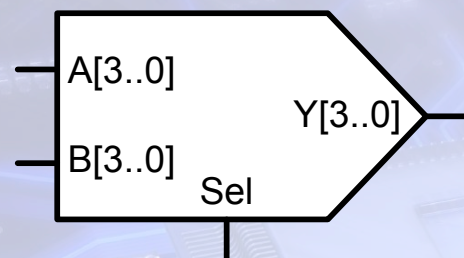
• Declaración de elementos.

- Es opcional.
- Declara señales intermedias, tipos de datos, componentes (diseño jerárquico), funciones, procedimientos y constantes.
- Son locales $\Rightarrow$ Sólo se pueden usar en la parte descriptiva de la arquitectura

• Statements concurrentes.

- Son obligatorios.
- Describen el comportamiento de la entidad en cualquiera de los estilos permitidos por VHDL:
 - ✓ Dataflow (Flujo de datos)
 - ✓ Behavioural (Funcional)
 - ✓ Structural (Estructural)

3. Estructura de un diseño básico en VHDL. Architecture (Arquitectura)



❖ Arquitectura correspondiente al mux de 2 a 1

```
Entity mux2a1_4bits is
  port(A, B : in bit_vector(3 downto 0);
        Sel : in bit;
        Y : out bit_vector(3 downto 0));
End mux2a1_4bits;
```

El nombre de la entidad debe indicarse aquí

Nombre de la arquitectura

Sección declarativa
No se declara
ningún elemento

```
Architecture Funcional of mux2a1_4bits is
```

```
Begin
```

```
--Descripción del comportamiento
```

Statements
concurrentes

```
Y <= A when Sel = '0'
  else B;
```

-- Y = A Si Sel es 0

-- de lo contrario (Sel = 1) Y = B

```
End Funcional;
```


3. Estructura de un diseño básico en VHDL. Architecture (Arquitectura)

❖ Descripción del mux de 2 a 1 de n bits

Entity mux2a1_nbits is

```
generic (Num_bits : integer := 16);      -- Por defecto mux de 16 bits

port(
    A : in bit_vector(Num_bits-1 downto 0);    -- Entrada de datos A
    B : in bit_vector(Num_bits-1 downto 0);    -- Entrada de datos B
    Sel : bit;                                -- Entrada de selección. El modo por defecto es in
    Y : out bit_vector(Num_bits-1 downto 0)     -- Salida de datos Y. Array de n bits
);
```

Architecture Funcional of mux2a1_4bits is

Begin

--Descripción del comportamiento

```
Y <= A when Sel = '0'      -- Y = A Si Sel es 0
    else B;                -- de lo contrario (Sel = 1) Y = B
```

End Funcional;

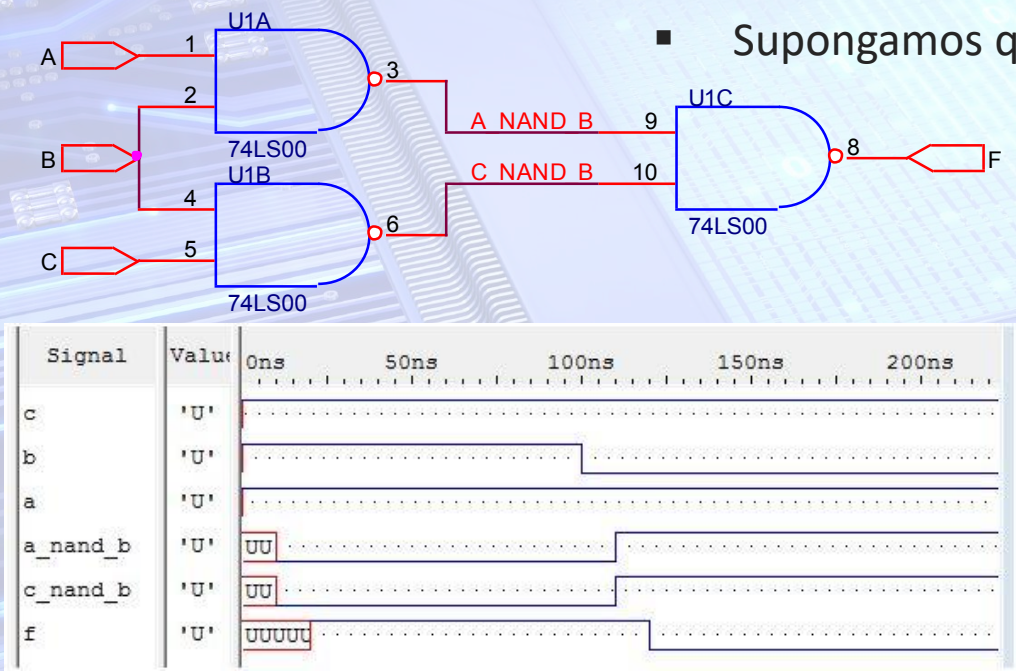
La descripción es la misma
indiferentemente del número de bits

4. Ejecución concurrente y secuencial.

Ejecución concurrente

❖ El funcionamiento de los sistemas digitales es paralelo

- Si cambia de valor alguna entrada, puede cambiar al mismo tiempo las señales de salida de dos o más componentes
- Eso implica que se tiene que evaluar el comportamiento de esos componentes al mismo tiempo
- Supongamos que se debe describir el siguiente circuito:



- Al cambiar de 1 a 0 la entrada B se debe evaluar la salida de las puertas U1A y U1B al mismo tiempo
- Como el cambio de B cambia el valor de las entradas de U1C, se debe evaluar su salida (F)
- Se dice que la ejecución del modelo de simulación del circuito debe ser concurrente

4. Ejecución concurrente y secuencial.

Ejecución concurrente

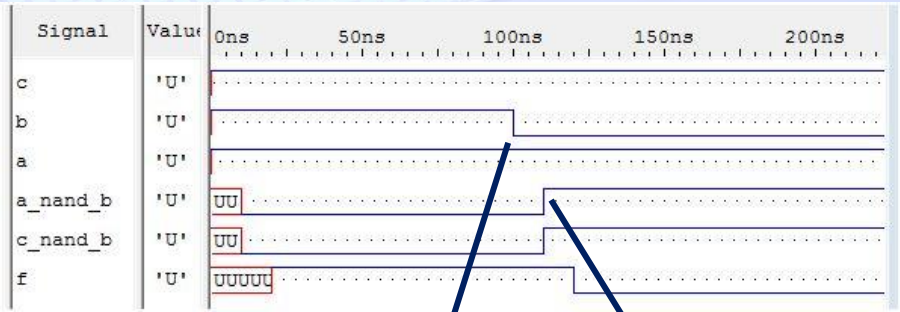
❖ Descripción VHDL del circuito.

```
Entity puertas is port(  
  a, b, c : in bit;  
  F : out bit);  
End puertas;  
Architecture Funcional of puertas is  
  Signal A_NAND_B, C_NAND_B : bit;  
  Begin  
    A_NAND_B <= A nand B;  
    C_NAND_B <= C nand B;  
    F <= A_NAND_B nand C_NAND_B;  
  End;
```

Declaración de señales

Área de ejecución concurrente

VHDL permite varios tipos de operadores, entre los que se incluyen los lógicos, como: **and, or, not, nand, nor y xor**



La simulación del circuito es temporal.

En VHDL también se puede indicar que la asignación del valor se realice después de un tiempo mediante la cláusula **after**

A_NAND_B <= A nand B after 10 ns;

El cambio de valor de b fuerza la evaluación de las instrucciones de asignación a A_NAND_B y C_NAND_B al mismo tiempo

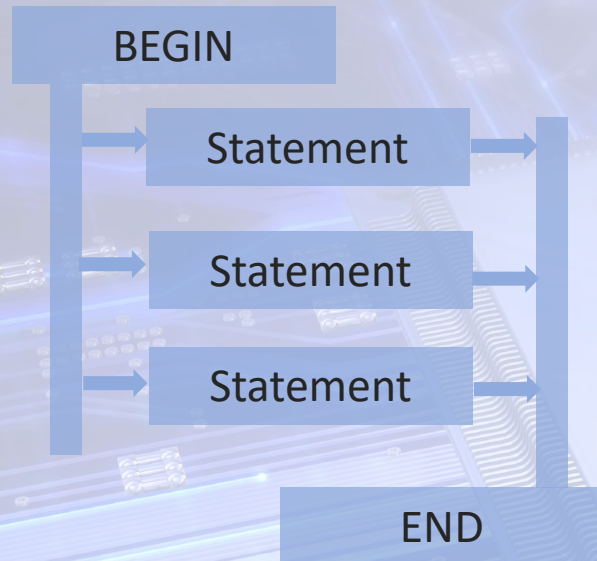
El cambio de valor de A_NAND_B y C_NAND_B fuerza la evaluación del statement de asignación a F

4. Ejecución concurrente y secuencial.

Ejecución concurrente

❖ Características de la ejecución concurrente

- El **área de concurrencia de la arquitectura** es la comprendida entre las palabras claves **Begin** y **End**
- Está compuesta por statements concurrentes.
- Todos se ejecutan en paralelo, si se produce un cambio de valor en alguna de sus señales de entrada.
- El orden de ejecución de los statements es indiferente del de escritura.
 - ✓ Estas dos descripciones son equivalentes



```

Architecture Funcional of puertas is
  Signal A_NAND_B, C_NAND_B : bit;
Begin
  A_NAND_B <= A nand B;
  C_NAND_B <= C nand B;
  F <= A_NAND_B nand C_NAND_B;
End;

```

```

Architecture Funcional of puertas is
  Signal A_NAND_B, C_NAND_B : bit;
Begin
  F <= A_NAND_B nand C_NAND_B;
  A_NAND_B <= A nand B;
  C_NAND_B <= C nand B;
End;

```


4. Ejecución concurrente y secuencial.

Ejecución secuencial

❖ Descripción de sistemas secuenciales

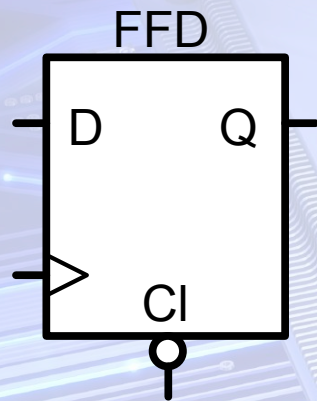
- La descripción de un circuito secuencial (Flip-flop) se tiene que realizar en una secuencia de pasos

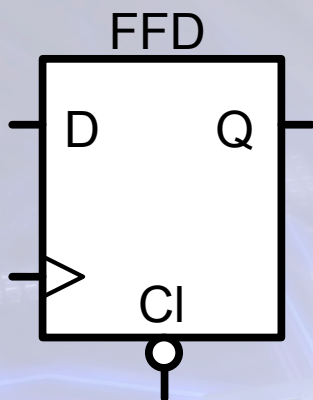
- Ejemplo. Flip-flop D con entrada de clear asíncrono**

- ✓ **La entrada Clear es prioritaria.** Por tanto, se evaluará primero.
- ✓ Si se produce algún cambio de valor en las señales de entrada **Clear o Clk**, se realiza la siguiente secuencia de pasos:
 - Se comprueba el valor de la entrada clear. Si es 0 se pone a 0 la salida del flip-flop.
 - Si Clear está a 1, se evalúa la señal de reloj.
 - Si se ha producido un flanco de subida en clk, $Q^{v+1} = D$
 - De lo contrario, no cambia de estado $Q^{v+1} = Q^v$

- Conclusión**

- ✓ Para describir el comportamiento del flip-flop se tiene que hacer secuencialmente.
- ✓ Una herramienta de síntesis tendrá que **ejecutar secuencialmente** el modelo descriptivo para poder interpretar que se trata de un **flip-flop D con entrada de clear asíncrono**.





4. Ejecución concurrente y secuencial. Ejecución secuencial

❖ Descripción del flip-flop D con entrada de clear asíncrono

Entity dff_clear is port(

Clk, Clear, D : in bit;

Q : out bit);

End dff_clear;

Architecture Funcional **of** dff_clear is

Begin

Process(Clk, Clear)

Begin

If Clear = '0' **then** --Empieza la ejecución

Q <= '0';

Elsif Clk'event **and** Clk = '1' **then**

Q <= D;

End if;

End process; --Termina la ejecución

End;

1. Se evalúa la entrada de clear:

- Si está a 0 $\Rightarrow Q = 0$ y se termina la ejecución
- De lo contrario se continúa

2. Se evalúa la señal de reloj:

- Si se ha producido un $\uparrow \Rightarrow Q = D$ y se termina la ejecución
- De lo contrario no se hace nada y termina la ejecución

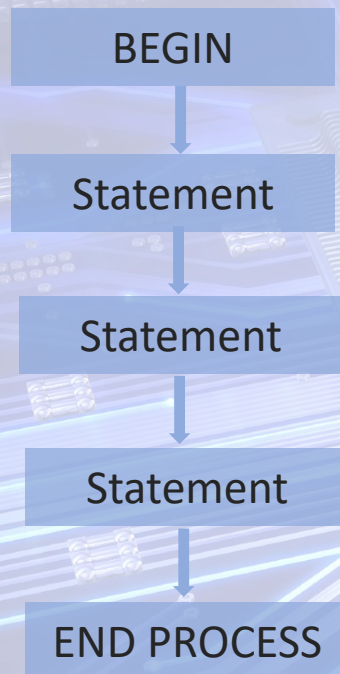
- En VHDL si no se le asigna un valor a una señal ésta mantiene su valor actual.
- Es la forma de describir elementos de memoria.

4. Ejecución concurrente y secuencial.

Ejecución secuencial

❖ Características de la ejecución secuencial

- Se realiza al ejecutarse un process (proceso), function (función) o procedure (procedimiento)
 - ✓ Se usan instrucciones secuenciales como el **IF THEN ELSE**.
 - ✓ También se usan statements de asignación de señal asociados o no a alguna instrucción secuencial.
- Los statements se ejecutan según el orden de escritura en el process de arriba a abajo
 - ✓ Se comienza con el primer statement que hay a continuación de **Begin**.
 - ✓ Si no se cambia el orden por alguna instrucción secuencial, se continúa con el **siguiente**.
 - ✓ Se termina la ejecución con el último statement (el situado antes de **END PROCESS**)
 - Se asigna un valor a una señal o variable, si se cumple la condición de la instrucción secuencial.
 - `Q <= '0';` --Clear = '0'
 - `Q <= D;` --Flanco de subida en Clk
 - La señal mantiene su valor si no se cumple la condición.



4. Ejecución concurrente y secuencial.

Ejecución secuencial

❖ Características de la ejecución secuencial

- El orden de escritura determina el comportamiento del process.
- Estos dos process son diferentes al intercambiar el orden de comparación

Entidad común

```
Entity codif is port(
  a, b : in bit;
  Y1, Y0 : out bit);
End codif;
```

Architecture Funcional of codif is
Begin

Process(a, b)

Begin

If b = '1' then

Y0 <= '0';

Y1 <= '1';

Elsif a = '1' then

Y0 <= '1';

Y1 <= '0';

Else

Y0 <= '0';

Y1 <= '0';

End if;

End process;

End;

Architecture Funcional of codif is
Begin

Process(a, b)

Begin

If a = '1' then

Y0 <= '1';

Y1 <= '0';

Elsif b = '1' then

Y0 <= '0';

Y1 <= '1';

Else

Y0 <= '0';

Y1 <= '0';

End if;

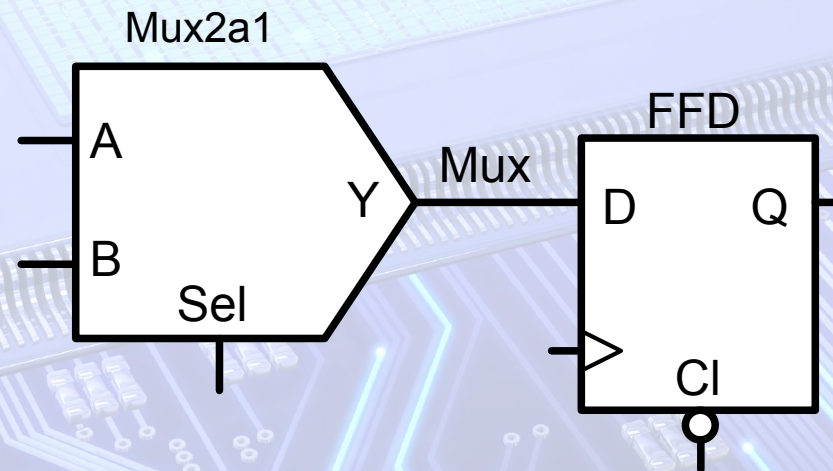
End process;

End;

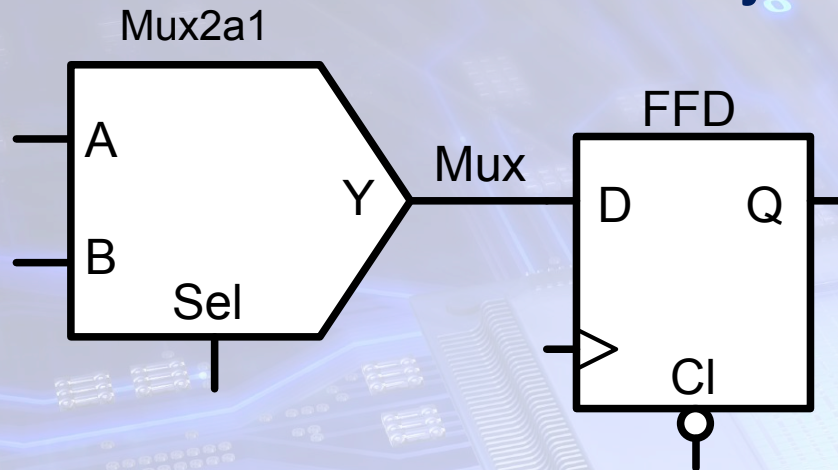
4. Ejecución concurrente y secuencial

❖ Todo el process en conjunto se considera un statement concurrente, aunque internamente la ejecución sea secuencial.

- El circuito de la figura se puede describir mediante dos statements concurrentes:
 - ✓ El multiplexor de 2 a 1 mediante un statement de asignación condicional de señal
 - ✓ El flip-flop mediante un process
 - ✓ Ambos se conectan mediante la señal Mux



4. Ejecución concurrente y secuencial



- Ambos estamentos se ejecutan en paralelo, si cambia de valor alguna de las entradas del componente.
 - a, b o Sel para Mux2a1
 - Clk o Clear para FFD
- El orden de escritura es indiferente.
- Ambos se conectan mediante la señal Mux.

❖ Descripción en VHDL

```
Entity Mux_dff is port(
    Clk, a, b, Sel, Clear : bit;
    Q : out bit);
End Mux_dff;
```

```
Architecture Funcional of Mux_dff is
    Signal Mux : bit;
```

```
Begin
```

```
    Mux <= a when Sel = '0' else b;           --statement concurrente
```

```
    -- El process en conjunto es un statement concurrente
```

```
    Process(Clk, Clear)                       --Inicio statement concurrente
```

```
    Begin
```

```
        If Clear = '0' then
```

```
            Q <= '0';
```

```
        Elsif Clk'event and Clk = '1' then
```

```
            Q <= Mux;
```

```
        End if;
```

```
    End process;
```

```
    --Fin statement concurrente
```

```
End;
```


5. Objetos en VHDL

❖ Concepto de Objeto

Elemento de VHDL que se emplea para representar y almacenar datos

❖ Tipos de objetos:

- **Signals** (señales)
- **Variables**
- **Constants** (constantes)

❖ Características generales:

- Cada objeto tiene un **tipo de dato** (ej. bit) y un **único conjunto de valores**
- **El tipo de dato determina los operadores** con los que se puede usar el objeto
- Se puede transferir el contenido de un objeto a otro, pero tienen que ser del **mismo tipo de datos**, o usar alguna **opción de conversión de tipo**, como se verá en otro tema posterior

5. Objetos en VHDL. Signals (señales)

❖ Concepto de Signal

- Son objetos que se usan para conectar elementos concurrentes (componentes, procesos y asignaciones concurrentes)
- Son similares a los hilos que conectan los componentes en un esquema eléctrico o una PCI

❖ Sintaxis

☼ Las signals son elementos de almacenamiento globales, respecto a la arquitectura

```
signal nom_signal {,nom_signal ...} : tipo_dato [:= valor_inicial];
```

- A la palabra clave **signal** le sigue uno o más nombres de señal
- Cada nombre crea una señal del tipo de dato indicado
- La señal se puede inicializar a un valor concreto mediante “ := ”

Ejemplo 1

Architecture Funcional **of** ejemplo1 **is**

```
    signal P1 : bit;
```

Begin

```
-- La declaración de P1 es local a la arquitectura  
-- por tanto sólo se puede usar en esta arquitectura  
End;
```

Ejemplo 2

Architecture Funcional **of** ejemplo2 **is**

```
    signal Vcc : bit := '1';
```

```
    signal Gnd : bit := '0';
```

Begin

```
-- La inicialización de señales no se suele usar en la síntesis  
-- Sólo se usa en la modelación y para definir el contenido de  
-- una RAM por algunas herramientas de síntesis  
End;
```


5. Objetos en VHDL. Signals (señales)

❖ Utilidad de las Señales

- Conectar dos statements concurrentes

Entity puertas **is port**(

a, b, c : **in** bit;

F : **out** bit);

End puertas;

Architecture Funcional **of** puertas **is**

Signal A_NAND_B, C_NAND_B : bit;

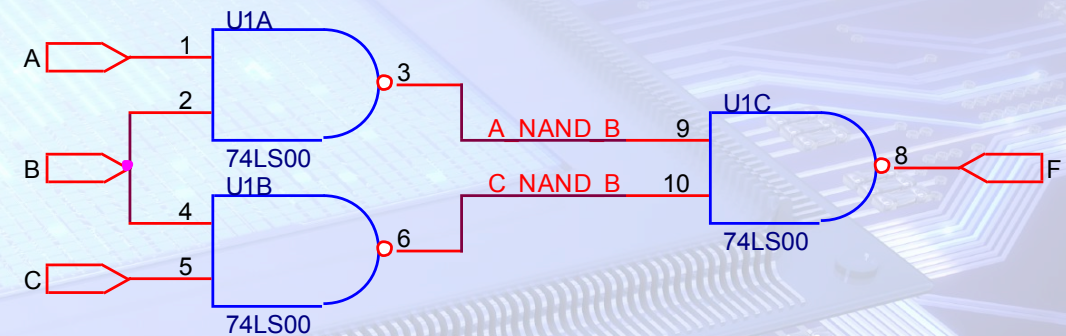
Begin

A_NAND_B <= A **nand** B;

C_NAND_B <= C **nand** B;

F <= A_NAND_B **nand** C_NAND_B;

End;



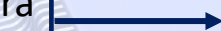
5. Objetos en VHDL. Signals (señales)

❖ Utilidad de las Señales

- Implementación de elementos de memoria
 - Se analizará en el tema de statements secuenciales

```
Entity sincro is port(  
  Clk, A, B : in bit;  
  Sal_sin : out bit);  
End sincro;  
Architecture Funcional of sincro is  
  Signal Q1, Q2 : bit;  
Begin  
  Process (Clk)  
  Begin  
    If Clk'event and Clk = '0' then  
      Q1 <= A;  
      Q2 <= B;  
    End if;  
  End Process;  
  Sal_sin <= Q1 and Q2;  
End;
```

Se genera un flip-flop D para cada señal, Q1 y Q2.



- Conectar unos componentes con otros en el estilo estructural
 - Se analizará en el tema de diseño jerárquico

5. Objetos en VHDL. Variables

❖ Concepto de Variable

- Son objetos que almacenan valores intermedios entre statements VHDL
- Están permitidas sólo en process, procedures y functions

❖ Sintaxis

```
variable nom_var {,nom_var ...} : tipo_dato [:= valor_inicial];
```

- A la palabra clave **variable** le sigue uno o más nombres de variables
- Cada nombre crea una variable del tipo de dato indicado
- La variable se puede inicializar a un valor concreto mediante “ := ”

- ⚡ Las variables son elementos de almacenamiento locales.
- ⚡ Sólo se pueden usar en el process que se declare.

```
Architecture Funcional of ejemplo1 is  
Begin
```

```
  Process
```

```
    variable A : bit;
```

```
    variable B : bit := '0';
```

```
  Begin
```

```
    -- Sólo se pueden usar en el process
```

```
  End process;
```

```
End;
```



5. Objetos en VHDL. Constants (constantes)

❖ Concepto de Constant

- Son objetos a los que se les asigna un valor al declararlos y éste no cambia durante la simulación

❖ Sintaxis

```
constant nom_constant {,nom_cons ...} : tipo_dato [:= valor_inicial];
```

- A la palabra clave **Constant** le sigue uno o más nombres de constantes
- Cada nombre crea una constante del tipo de dato indicado
- Lo normal es inicializarla a un valor concreto mediante “ := ”
- Si no se inicializa se llama **constante diferida**:
 - ✓ Se declara en un **package**
 - ✓ Se le asigna el valor en el **package body**

❖ Ejemplos

```
constant Fin_cuenta : bit_vector(7 downto 0) := X"AF";
```

```
constant Total : integer := 90;
```


5. Objetos en VHDL. Constants (constantes)

❖ Utilización de las Constantes

- Documentar el diseño
- Facilitar la modificación del diseño

❖ Ejemplo. Decodificador de direcciones

Entity comparador **is port**(

A : **in** bit_vector(3 **downto** 0);

CS_RAM : **out** bit);

End comparador;

Architecture Funcional **of** comparador **is**

Constant Dir_RAM : bit_vector(3 **downto** 0) := "0101";

Begin

--Descripción del comportamiento

CS_RAM <= '1' **when** A = DIR_RAM
 else '0';

-- El chip select se activa si A es igual a la

-- dirección de la RAM. Para cambiar el valor
-- basta con cambiar el valor de la constante

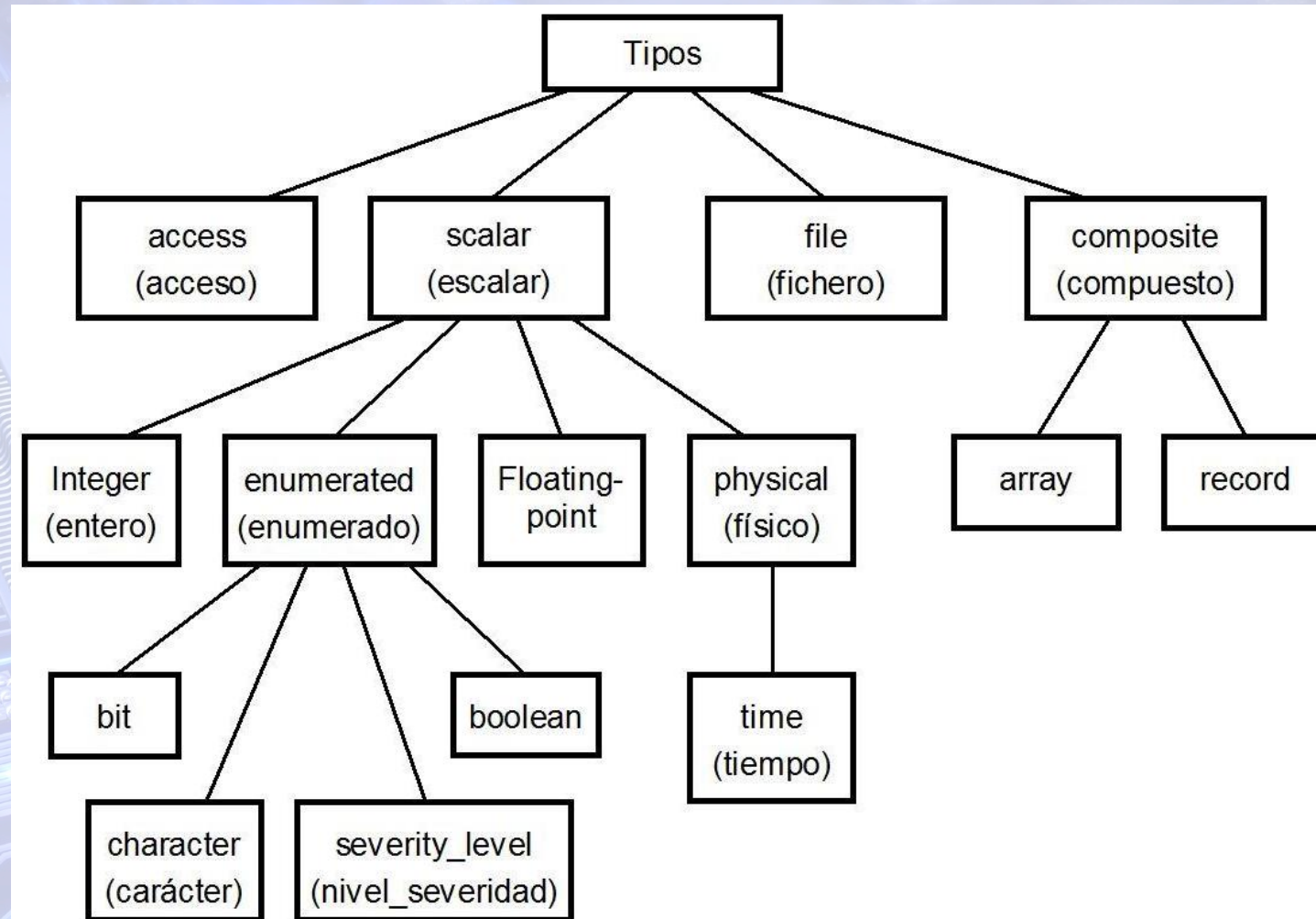
End Funcional;



6. Tipos de datos básicos

Tipos de datos definidos en el estándar 1076-87

- ⚙ Estos tipos de datos se incluyen en el Package Standard, que está incluido en la Librería std
- ⚙ No hace falta hacer visible ni el package ni la librería para usarlos



6. Tipos de datos básicos. Tipos de datos definidos en el estándar 1076-87. Escalar

❖ Características del tipo escalar

- Representan un único valor y están ordenados por su valor
 - ✓ Por lo tanto, se pueden usar con los operadores relacionales, tales como mayor, menor, igual, etc.

❖ Tipo integer (entero)

- Representa los valores enteros dentro de un rango dado
- El estándar especifica un rango mínimo de 32 bits:
 - ✓ -2147483647 a 2147483647
- Existen subtipos predefinidos en el estándar que restringen el rango:
 - ✓ **Natural.** Desde el 0 hasta 2147483647
 - ✓ **Positive.** Igual que el natural excluyendo el 0.
- Ejemplo:
`signal Num_bits : integer;`
- Al definir un objeto se puede restringir el rango.
`signal Num_8bits : integer range 0 to 255;`

6. Tipos de datos básicos. Tipos de datos definidos en el estándar 1076-87. Escalar

❖ Tipo Real

- Representa los números reales
- Deben contener un punto decimal
- Se puede usar la representación científica
- El rango mínimo especificado: **-1.0E38 a +1.0E38**
- Ejemplo:

```
signal F0: real := 4569.3;
```

```
SIGNAL a : real;
```

```
a <= 1.0;                --Ok
```

```
a <= 1;                  --error
```

```
a <= -1.0E10;            --Ok
```

```
a <= 1.5E-20;            --Ok
```

```
a <= 5.3 ns;             --error tipo físico
```


6. Tipos de datos básicos. Tipos de datos definidos en el estándar 1076-87. Escalar

❖ Tipo Enumerated (enumerado)

- Se usa para definir la mayoría de los tipos de datos estándar de VHDL, y permite al usuario crearse sus propios tipos de datos
- Los valores del tipo enumerado se indican mediante una lista

■ Sintaxis

- ✓ Se usa el statement de declaración de tipo de dato

```
Type nom_tipo is (identificador | literal_char {,.....} );
```

- ✓ Los literales del tipo carácter son letras o números encerrados entre comillas simples: '0' 'A' 'Z'

- **Aplicaciones:** Definir nuevos tipos de datos

-- Se define un tipo de datos de 4 valores

```
type cuatro_val is ('X', '0', '1', 'Z');
```

-- 'X' desconocido

-- '0' valor lógico 0 o falso

-- '1' valor lógico 1 o verdad

-- 'Z' Valor Alta impedancia

```
entity gastronomica is  
type comida is (Desayuno, Almuerzo, Cena, Tapas_nocturna);  
port (comida_anterior: in comida;  
       proxima_comida: out comida)  
end gastronomica;  
architecture comidas of gastronomica is  
begin  
    with comida_anterior select  
    proxima_comida <= Desayuno when Cena | Tapas_nocturna;  
                      Almuerzo when Desayuno;  
                      Cena when Almuerzo;  
end comidas;
```


6. Tipos de datos básicos. Tipos de datos definidos en el estándar 1076-87. Escalar

❖ Tipo Enumerated (enumerado)

▪ Aplicaciones

✓ Describir una máquina de estados

- El tipo enumerado se usa para representar los estados del autómata

💡 El orden en el que se lista los valores define el orden léxico para este tipo

architecture FSM of VCONTROL is

type estados is (Inicial, espera, muestreo, visualiza);

signal estado_actual, estado_sig : estados;

begin

....

--Transición de estados

STTANS: **process** (estado_actual, Modo)

begin

case estado_actual is

when Inicial =>

....

when espera =>

....

....

end case;

end process;

end FSM;

6. Tipos de datos básicos. Tipos de datos definidos en el estándar 1076-87. Escalar

❖ Tipo Character (carácter)

- Es similar al tipo carácter de los lenguajes de programación
- Es un tipo enumerado que define los caracteres del código ASCII:

💡 Un literal carácter está encerrado entre comillas simples

Ejemplo

Variable eje_caracter : character := 'P';

nul,	soh,	stx,	etx,	eot,	enq,	ack,	bel,
bs,	ht,	lf,	vt,	ff,	cr,	so,	si,
dle,	dc1,	dc2,	dc3,	dc4,	nak,	syn,	etb,
can,	em,	sub,	esc,	fsp,	gsp,	rsp,	usp,
' ','!',	'"',	'#',	'\$',	'%',	'&',	'"',	
'(', ')',	'*',	'+',	','	'-',	':',	'/',	
'0',	'1',	'2',	'3',	'4',	'5',	'6',	'7',
'8',	'9',	':',	';',	'<',	'=',	'>',	'?',
'@',	'A',	'B',	'C',	'D',	'E',	'F',	'G',
'H',	'I',	'J',	'K',	'L',	'M',	'N',	'O',
'P',	'Q',	'R',	'S',	'T',	'U',	'V',	'W',
'X',	'Y',	'Z',	'[',	'\',	']',	'^',	'_',
'`',	'a',	'b',	'c',	'd',	'e',	'f',	'g',
'h',	'i',	'j',	'k',	'l',	'm',	'n',	'o',
'p',	'q',	'r',	's',	't',	'u',	'v',	'w',
'x',	'y',	'z',	'{',	' ',	'}',	'~',	del);

6. Tipos de datos básicos. Tipos de datos definidos en el estándar 1076-87. Escalar

❖ Tipo Boolean

- Es un tipo enumerado con dos valores: **True y False**
- Representa el resultado de un test condicional realizado con los operadores lógicos (and, or, etc) o relacionales (=, /=, <, >, etc)
- **Ejemplo:**
 - Se define dos constantes de tipo boolean
 - Constant** verdad : boolean := true;
 - Constant** falso : boolean := false;

❖ Tipo Bit

Está definido como:
Type bit is ('0', '1');

- Es un tipo enumerado que representa los valores binarios
- Los posibles valores son: '0' y '1'. Se definen como tipo character
- Es la representación básica de un hilo o una señal binaria
- **Ejemplo:**
 - Se define dos señales de tipo bit
 - Signal** Clk : bit;
 - Signal** Reset : bit;

6. Tipos de datos básicos. Tipos de datos definidos en el estándar 1076-87. Escalar

❖ Tipo Physical (físico)

- Se usan para representar magnitudes físicas del mundo real: longitud, tiempo, voltaje, corriente, etc.
- El más usado es el tipo **Time (tiempo)**
- Se caracterizan por usar una unidad de medida
- Se definen a partir de una **unidad base**, que en el tipo time es el **femtosegundo**
- **Definición del tipo time**

Type time is range -2147483647 to 2147483647

units

fs; --femtosegundo
ps = 1000 fs; --picosegundo
ns = 1000 ps; --nanosegundo
us = 1000 ns; --microsegundo
ms = 1000 us; --milisegundo
sec = 1000 ms; --segundo
min = 60 sec; --minuto
hr = 60 min; --hora

end units;

Ejemplo:

Constant periodo : time := 100 ns;

⚡ Se debe incluir un espacio en blanco entre el valor y la unidad

6. Tipos de datos básicos. Tipos de datos definidos en el estándar 1076-87. Compuestos

❖ Tipo compuesto

- Son grupos de uno o más tipos de datos
- Se distinguen dos tipos:
 - ✓ **Tipo array**
 - ✓ **Tipo record.** Sólo se hará una introducción.

❖ Tipo Array

- Es un conjunto de elementos del mismo tipo de datos
- El estándar incluye tipos array como el **bit_vector**, pero el usuario también puede crearse sus propios tipos de datos array
- **Sintaxis:**

Type nom_tipo **is array** (expres **to** | **downto** expres) **of** tipo_elem;

- **Ejemplos**

type num_16bits **is array**(15 **downto** 0) **of** bit; -- orden descendente

type palabra_16bits **is array**(0 **to** 15) **of** bit; -- orden ascendente

- ✓ Una vez declarado el tipo de dato se puede declarar un objeto de este tipo

signal bus : palabra_16bits;

constant dato : num_16bits := X"55AA";

6. Tipos de datos básicos. Tipos de datos definidos en el estándar 1076-87. Compuestos

❖ Tipo Array

- Hay arrays de varias dimensiones:

0

Escalar

0 1 0 0 0

Array 1D

0 1 0 0 0

1 0 0 1 0

1 1 0 0 1

Array 1Dx1D

0 1 0 0 0

1 0 0 1 0

1 1 0 0 1

Array 2D

- El **array 1D** es un vector: conjunto de bits
- El **array 1Dx1D** es un array de vectores: se puede usar para definir la matriz de celdas de memoria de una ROM o una RAM
- El **array 2D** es un array de escalares
- Ejemplo de array **1Dx1D**
 - ✓ Se define la matriz de una RAM de 8 x 4

type palabra **is** array (3 **downto** 0) **of** bit;

--array 1D = palabra de 4 bits

type matriz **is** array (0 **to** 7) **of** palabra;

--array 1Dx1D = matriz

signal Memoria : matriz;

6. Tipos de datos básicos. Tipos de datos definidos en el estándar 1076-87. Compuestos

❖ Tipo Array

▪ Ejemplo de array 2D

- ✓ Se define la matriz de una RAM de 8 x 4
type matriz2D **is array** (0 to 7, 3 downto 0) **of** bit;
signal Memoria : matriz;

▪ Tipo array más común definido en el estándar 1076-87

✓ Bit_vector

- Se define como un conjunto de bits no restringido
type bit_vector **is array**(natural range <>) **of** bit;
 - No se especifica completamente el rango al definirlo: los índices están limitados por el rango del tipo natural
 - Lo indica el diseñador al declarar un objeto de este tipo
- Permite representar un bus de señales digitales de dos estados y números binarios
- **Ejemplos.** Se declaran varios objetos del tipo bit_vector
Signal Num_16bits : bit_vector(15 **downto** 0); -- descendente
Signal Num_8bits : bit_vector(0 **to** 7); -- ascendente

💡 Descripción de la matriz de una RAM 8 x 4 mediante bit_vector

```
type matriz is array (0 to 7) of  
  bit_vector(3 downto 0);  
signal Memoria : matriz;
```


6. Tipos de datos básicos. Tipos de datos definidos en el estándar 1076-87. Compuestos

❖ Inicialización de array 1D (bit_vector)

- Usando un literal del tipo de datos del array

- ✓ Se denomina literal cadena de bits
- ✓ Se encierra entre comillas dobles
- ✓ Se puede especificar la base poniendo una letra delante de las comillas de apertura:

- **B** indica que es un **número binario**
- **O** indica que es un **número octal**
- **X** indica que es un número **hexadecimal**
- Si no se indica ninguna letra se considera base 2

- ✓ **Ejemplos.** Supongamos que A es un array de 8 bits

A <= "00111010"; --Número binario

A <= X"3A"; --Número hexadecimal

A <= O"72"; --Número octal. Erróneo sólo 6 bits

6. Tipos de datos básicos. Tipos de datos definidos en el estándar 1076-87. Compuestos

❖ Tipo Array

▪ Acceso a los elementos del array

- ✓ Se usa un índice que es un valor entero
- ✓ El índice se indica entre paréntesis a la derecha del objeto de tipo array
 - **A(0) A(7)**
- ✓ El índice especifica la posición del elemento al que se accede. La posición depende de cómo se haya declarado el array
 - Array descendente: 7 **downto** 0
 - El elemento 7 es el situado más a la izquierda
 - El elemento 0 es el situado más a la derecha
 - Array ascendente: 0 **to** 7
 - El elemento 7 es el situado más a la derecha
 - El elemento 0 es el situado más a la izquierda
 - **Ejemplos.** Supongamos que A es un array de 8 bits y B un bit

Type num_8bits **is** array (7 **downto** 0) **of** bit;

signal A : num_8bits;

A <= "00111010"; --Número binario

B <= A(1); --B = '1'

B <= A(7); --B = '0'

B <= A(5); --B = '1'

💡 También se puede describir la señal A como un bit_vector
signal A : bit_vector(7 downto 0);

6. Tipos de datos básicos. Tipos de datos definidos en el estándar 1076-87. Compuestos. Tipo Array

❖ Transferencia del contenido de un array a otro

- Cuando se transfiere el contenido de un array a otro deben ser del mismo tipo y del mismo tamaño
- Si tienen distinto tamaño se puede transferir el contenido accediendo a varios elementos de un array al mismo tiempo mediante el operador de rango **downto** o **to**

✓ Ejemplo. Array de 4 bits y array de 8 bits

```
Signal A_4b : bit_vector(3 downto 0) := "1010";
```

```
Signal B_8b : bit_vector(7 downto 0) := "01110101";
```

```
A_4b <= B_8b;           -- Erróneo
```

```
A_4b <= B_8b(5 downto 2);      --Ok A_4b = "1101"
```

- Se puede transferir indiferentemente del orden de declaración

✓ Ejemplo:

```
Signal A_asc : bit_vector(0 to 3) := "1010";
```

```
Signal B_des : bit_vector(3 downto 0) := "0101";
```

```
A_asc <= B_des;          -- A_asc = "0101".
```

Es equivalente a:

```
-- A_asc (0) <= B_des(3), A_asc (1) <= B_des (2), A_asc (2) <= B_des(1), A(3) <= B_des(0)
```


6. Tipos de datos básicos. Tipos de datos definidos en el estándar 1076-87. Compuestos. Tipo Array

❖ Inicialización de array 1D

■ Asignación agregada

- ✓ Se indica los valores del array entre paréntesis separados por comas
- ✓ Hay varias opciones.

- **Asociación posicional**

- Se listan los valores en el orden correspondiente a la estructura del array

- **Asociación por nombres**

- Se indica el índice del elemento y su valor mediante el operador =>

- **Sintaxis**

(índice | rango_índices | **others** => valor)

- **Others** permite inicializar un array con todos los bits al mismo valor indiferentemente de su tamaño

Ejemplos:

Signal A : bit_vector(3 downto 0);

Signal B : bit_vector(0 to 3);

-- Asociación posicional

A <= ('0','1','0','1');

-- A(3)= '0', A(2) = '1', A(1) = '0', A(0) = '1'

B <= ('0','1','0','1');

-- B(0) = '0', B(1) = '1', B(2) = '0', B(3) = '1'

-- Asociación por nombres

A <= (**others** => '0'); -- A = "0000"

B <= (3 => '1', 2 => '1', 1 => '0', 0 => '1');

-- Es equivalente a B <= "1011"

A <= (1 => '0', **others** => '1');

-- Es equivalente a A <= "1101"

B <= (0 to 1 => '0', 2 to 3 => '1');

-- Es equivalente a B <= "0011"

6. Tipos de datos básicos. Tipos de datos definidos en el estándar 1076-87. Compuestos

❖ Tipo Array

▪ Inicialización de array bidimensionales 1Dx1D

❖ Acceso a los elementos de un array bidimensional

▪ Acceso a un vector (palabra)

- ✓ Se usa el índice del vector = dirección de la palabra de la ROM

- ✓ Lectura de la palabra 3

Registro <= rom(3);

-- Registro = "0111"

▪ Acceso a un bit de un vector

- ✓ Se usa el índice del vector y el del bit:

Rom(índice_palabra) (índice_bit)

- ✓ Lectura del bit 2 de la palabra 3

Registro(2) <= rom(3) (2);

-- Registro(2) = '1'

- ✓ Se usa la **asignación agregada**

type palabra **is** array(3 **downto** 0) **of** bit;

type mem_data **is** array(0 **to** 7) **of** palabra;

signal registro : palabra;

constant rom : mem_data :=

(('0', '0', '0', '0'), --palabra 0

('0', '1', '1', '0'), --palabra 1

('1', '1', '0', '0'), --palabra 2

('0', '1', '1', '1'), --palabra 3

('0', '1', '1', '0'), --palabra 4

('1', '0', '0', '0'), --palabra 5

('0', '1', '1', '0'), --palabra 6

('0', '0', '0', '0')); --palabra 7

6. Tipos de datos básicos. Tipos de datos definidos en el estándar 1076-87. Compuestos. Tipo Record

❖ Tipo Record

- Un objeto de tipo record tiene varios elementos de diferentes tipos de datos
- Consta de varios campos
- Cada campo lleva asociado un nombre y un tipo de datos
- Los campos individuales de un record se referencian por su nombre
- **Ejemplo:**
 - ✓ Se declara el tipo de datos **Formato_inst** del tipo **record**
 - ✓ Se declaran dos objetos del tipo de datos **Formato_inst**
 - ✓ Se muestran varios ejemplos de asignaciones usando record
 - ✓ No lo contemplan todas las herramientas de síntesis

Ejemplo. Formato de instrucción de un procesador

Type otype is (add, sub, mul, div, jmp);

Type Formato_inst is record

Cod_op : otype;

Fuente : bit_vector(3 **downto** 0);

Destino : bit_vector(3 **downto** 0);

End record;

Variable Inst_A, Inst_B : Formato_inst;

Variable Reg_inst : otype;

Variable Reg_fuente, Reg_destino : bit_vector(3 **downto** 0);

Inst_A := (add, X"1", X"3");

Inst_B := Inst_A;

Reg_inst := Inst_B.cod_op; -- Reg_inst = add

Reg_inst := Inst_A.Fuente; -- Error

Reg_destino := Inst_A.Fuente; -- Reg_destino = X"1"

Inst_A.destino := Reg_destino; -- (add, X"1", X"1")

6. Tipos de datos básicos. Tipos de datos definidos en el estándar 1076-87. Estándar IEEE 1164

❖ ESTÁNDAR IEEE 1164

- Este estándar fue publicado para cubrir las limitaciones de los tipos de datos del estándar 1076-87
- Los tipos de datos primitivos no permitían representar múltiples valores para un hilo (alta impedancia, desconocido, etc), que son bastante importantes, tanto en la simulación como en la síntesis
- Esto obligó a que cada fabricante software definiese sus propios tipos de datos mediante tipos enumerated
 - ✓ Se crearon múltiples tipos de datos no estandarizados
 - ✓ Esto limitó la portabilidad de los diseños
- Para solucionar este inconveniente, se publicó el **estándar 1164**, que define un tipo de dato de nueve valores, **std_logic**

6. Tipos de datos básicos. Tipos de datos definidos en el estándar 1076-87. Estándar IEEE 1164

❖ ESTÁNDAR IEEE 1164

- Se declara como tipo de dato base el `std_ulogic`
- Se declara como un tipo enumerado:

```
Type std_ulogic is ( 'U',      --No definido  
                    'X',      --Forzando desconocido  
                    '0',      --Forzando 0 lógico  
                    '1',      --Forzando 1 lógico  
                    'Z',      --Alta impedancia  
                    'W',      --Desconocido weak  
                    'L',      --0 weak  
                    'H',      --1 weak  
                    '-',      --Indiferente  
                    );
```

💡 Los valores indicados por letras se declaran como caracteres de letras mayúsculas

6. Tipos de datos básicos. Tipos de datos definidos en el estándar 1076-87. Estándar IEEE 1164

❖ ESTÁNDAR IEEE 1164

- A partir del **std_ulogic** se declara el tipo **std_logic**
- Este se declara como un tipo de datos resuelto
 - ✓ En el tema correspondiente a los statements concurrentes se analizará este concepto
- **Tabla de valores:**

 **Declaración de std_logic**

```
subtype std_logic is resolved std_ulogic;
```

Valor	Descripción
'U'	No definido: No inicializado o no definido
'X'	Desconocido (forzado)
'0'	0 lógico (forzado)
'1'	1 lógico (forzado)
'Z'	Alta impedancia
'W'	Desconocido (weak)
'L'	0 lógico (weak)
'H'	1 lógico (weak)
'-'	Indiferente

6. Tipos de datos básicos. Tipos de datos definidos en el estándar 1076-87. Estándar IEEE 1164

❖ Tipo array

- El Estándar IEEE 1164 declara el tipo **std_logic_vector**
 - ✓ Se declara como un array no restringido de std_logic
 - ✓ El rango de los índices se restringe al del tipo de dato natural

❖ Requisitos para usar el package del estándar IEEE 1164

- Se debe añadir dos statements al fichero fuente VHDL
 - Library ieee;**
 - Use ieee.std_logic_1164.all;**
- **Normas**
 - ✓ **Library ieee;**
 - Carga la librería estándar IEEE 1164, denominada ieee
 - ✓ **Use ieee.std_logic_1164.all;**
 - Hace visible todos los elementos del package std_logic_1164 de la librería ieee
 - ✓ Se debe poner los dos statements al principio del fichero fuente y repetir el statement use antes de cada unidad de diseño, que lo emplee (pares entity y architecture), para hacer visible el contenido de la librería estándar a esa unidad de diseño



6. Tipos de datos básicos. Tipos de datos definidos en el estándar 1076-87. Estándar IEEE 1164

- En lo sucesivo se usará los tipos **std_logic** y **std_logic_vector** en la descripción de los diseños:
 - ✓ Declaración de puertos, señales, constantes y variables
 - ✓ De esta forma se tiene una descripción portable
- El tipo **std_logic_vector** es un conjunto de **std_logic**, que representan los caracteres '1' y '0' entre otros, pero éstos no tienen ningún peso asociado como el sistema binario
 - ☹ No se pueden usar directamente con los operadores aritméticos
 - ☺ En el tema 4 se analizarán las alternativas

❖ Ejemplo. Descripción del mux de 2 a 1 usando los tipos std_logic y std_logic_vector

```
-- Mux de 2 a 1 de 4 bits
-- Primero se hace visible el package
Library ieee;
Use ieee.std_logic_1164.all;
-- Ahora se puede usar los tipos std_logic y std_logic_vector
```

Entity mux2a1_4bits is

```
port(A, B : in std_logic_vector(3 downto 0);
      Sel : in std_logic;
      Y : out std_logic_vector(3 downto 0) );
```

```
End mux2a1_4bits;
```

Architecture Funcional of mux2a1_4bits is

Begin

```
--Descripcion del comportamiento
```

```
Y <= A when Sel = '0' -- Y = A Si Sel es 0
    else B;           -- de lo contrario (Sel = 1) Y = B
```

```
End Funcional;
```


7. Operadores básicos en síntesis

❖ Operadores

- **VHDL proporciona varios tipos de operadores predefinidos:**

- ✓ Operadores de asignación
- ✓ Operadores lógicos
- ✓ Operadores aritméticos
- ✓ Operadores relacionales
- ✓ Operadores de desplazamiento
- ✓ Operadores de concatenación

- **Características comunes:**

- ✓ Cada operador sólo se puede usar con unos tipos de datos concretos
- ✓ El tipo de dato generado como resultado de un operador puede diferir del que emplea como entrada la operación



7. Operadores básicos en síntesis

❖ Operadores de asignación

- Se usan para asignar valores a los puertos, señales, variables y constantes
- **Tipos:**
 - <=** Asigna un valor a una señal o a un puerto
 - :=** Asigna un valor a un variable, constante o generic, o el valor inicial a los objetos
 - =>** Se usa en la asignación agregada para asignar valores a elementos individuales de un vector o con **others**
- **Ejemplos**

Declaración de objetos del ejemplo

```
signal X : std_logic;  
variable Y : std_logic_vector(3 downto 0);  
signal W : std_logic_vector(0 to 7);
```

Asignaciones legales

```
X <= '1';  
Y := "0000";  
W <= "10000000";      W <= X"80";  
W <= (0 => '1', others => '0');  
-- Las tres asignaciones a W son equivalentes
```


7. Operadores básicos en síntesis

❖ Operadores lógicos

- Se usan para describir funciones lógicas y operaciones lógicas (procesamiento de bit)
- Tipos:

Operador	Descripción	Tipos de operandos	Tipo resultado
And	And	Bit, bit_vector, std_logic, std_logic_vector, boolean, o array 1-D de boolean	Mismo tipo
Or	Or		
Nand	Nand		
Nor	Nor		
Xor	Or exclusiva		
xnor	Nor exclusiva		

💡 El operador XNOR se introdujo en el estándar del 93

Ejemplos de operadores lógicos

W <= **NOT** a **AND** b; -- (a' . b)

W <= **NOT** (a **AND** b); -- (a . b)'

W <= a **NAND** b; -- (a . b)'

7. Operadores básicos en síntesis

❖ Operadores relacionales

- Se usan para comprobar los valores relativos de dos tipos escalares
- El resultado es siempre un valor booleano: true o false
- **Tipos:**

Operador	Descripción	Tipos de operandos	Tipo resultado
=	Igual a	Cualquier tipo escalar o array de cualquier tipo discreto (generalmente bit_vector, std_logic_vector)	Boolean
/=	Distinto a		
<	Menor que		
>	Mayor		
<=	Menor o igual que		
>=	Mayor o igual que		

Ejemplo de operador relacional

Igual <= '1' when A = B else '0';

Mayor_Igual <= '1' when A >= B else '0';

7. Operadores básicos en síntesis

❖ Operadores aritméticos

- Se usan para realizar operaciones aritméticas
- Tipos:

Operador	Descripción	Tipos de operandos	Tipo resultado
+	Suma	Cualquier tipo numérico	Mismo tipo
-	Resta		
*	Multiplicación	Entero, real o tipo físico	Mismo tipo Físico si alguno es de este tipo
/	División		
**	Exponenciación	Izq. Entero o real Der. Entero	Mismo izquierda
MOD	Módulo	Entero	Entero
REM	Resto		
ABS	Valor absoluto	Cualquier tipo numérico	Mismo tipo

7. Operadores básicos en síntesis

❖ Operadores de desplazamiento

- Se usa para el desplazamiento y rotación de datos
- Se introdujeron en el estándar 1076-93
- **Tipos:**

Operador	Nombre	Tipos de operandos	Tipo resultado
sll	Desplazamiento lógico a la izquierda	Izq. Cualquier tipo de array de bit o boolean (generalmente bit_vector) Der. Entero: indica el número de desplazamientos	Mismo que el de la izquierda
srl	Desplazamiento lógico a la derecha		
sla	Desplazamiento aritmético a la izquierda		
sra	Desplazamiento aritmético a la derecha		
rol	Rotación a la izquierda		
ror	Rotación a la derecha		

💡 El tipo std_logic_vector no se puede usar directamente con estos operadores

- ☹ No se ha sobrecargado estos operadores en el package std_logic_1164
- 😊 En el tema 4 se analizarán las alternativas

7. Operadores básicos en síntesis

❖ Operadores de desplazamiento

- Descripción de las operaciones:

Operador	Nombre	Descripción
sll	Desplazamiento lógico a la izquierda	Se desplaza el contenido a la izquierda. Se pierde el bit situado más a la izquierda y al bit de la derecha se le asigna un '0'
srl	Desplazamiento lógico a la derecha	Se desplaza el contenido a la derecha. Se pierde el bit situado más a la derecha y al bit de la izquierda se le asigna un '0'
sla	Desplazamiento aritmético a la izquierda	Se desplaza el contenido a la izquierda. Se pierde el bit situado más a la izquierda y se repite el bit situado más a la derecha
sra	Desplazamiento aritmético a la derecha	Se desplaza el contenido a la derecha. Se pierde el bit situado más a la derecha y se repite el bit situado más a la izquierda
rol	Rotación a la izquierda	Se desplaza el contenido a la izquierda. El bit que sale por la izquierda se introduce por la derecha
ror	Rotación a la derecha	Se desplaza el contenido a la derecha. El bit que sale por la derecha se introduce por la izquierda



7. Operadores básicos en síntesis

❖ Operador de concatenación

- Se representa por &
- Se usa para:
 1. Unir dos array unidimensionales (1-D) y formar un array de dimensión igual a la suma de la dimensión de cada uno
 2. Unir un array 1-D y uno o más elementos escalares del tipo del array y formar un array mayor
 3. Unir dos o más escalares del mismo tipo de datos para formar un array

Declaración de objetos del ejemplo

```
variable X, Y : std_logic;  
variable A_4b : std_logic_vector(3 downto 0);  
variable A_8b : std_logic_vector(0 to 7);  
variable A_2b : std_logic_vector(0 to 1);  
variable A_12b : std_logic_vector(0 to 11);
```

Ejemplos

```
X := '1';  
Y := '0';  
A_4b := "0110";  
A_8b := "10010001";  
A_2b := X & Y;           -- A_2b = "10"  
A_4b := A_4b(1 downto 0) & Y & X;   -- A_4b = "1001"  
A_12b := A_4b & A_8b;       -- A_12b = "1001_10010001"
```


7. Operadores básicos en síntesis

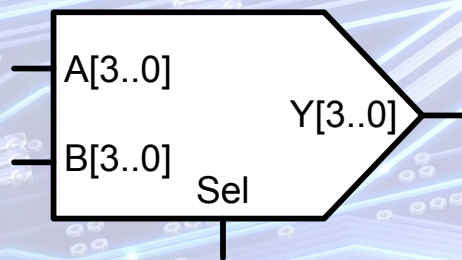
❖ Orden de precedencia de los operadores

- Los operadores de la misma prioridad se evalúan de izquierda a derecha en la expresión
- Los paréntesis se usan para cambiar el orden

Mayor	**	ABS	NOT				
	*	/	MOD	REM			
	+	-	&				
	=	/=	<	<=	>	>=	
Menor	AND	OR	NAND	NOR	XOR	XNOR	

8. Estilos de descripción

- ❖ Hay tres estilos distintos para describir la arquitectura
 - **Dataflow (flujo de datos)**
 - **Behavioral (Comportamiento)**
 - **Estructural**
- ❖ Se puede usar en una misma arquitectura varios estilos descriptivos
- ❖ **Estilo flujo de datos**
 - Se describe el circuito indicando como se mueve los datos a través del sistema
 - Se usan instrucciones concurrentes de asignación de valor a señales
 - **Descripción de un mux de 2 a 1 de 4 bits**



Architecture Funcional of mux2a1_4bits is

Begin

--Descripcion del comportamiento

Y <= A **when** Sel = '0'

else B;

End Funcional;

-- Y = A Si Sel es 0

-- de lo contrario (Sel = 1) Y = B

8. Estilos de descripción

❖ Estilo behavioral

- Es el estilo descriptivo de más alto nivel de VHDL
- Se describe el circuito indicando las funciones que realiza a lo largo del tiempo
- Se suele usar para describir máquinas de estado y algoritmos
- Se usan procesos
- **Descripción de un mux de 2 a 1 de 4 bits**

Architecture Funcional of mux2a1_4bits is

Begin

Process(A, B, Sel)

Begin

If Sel = '0' then

Y <= A; -- Y = A Si Sel es 0

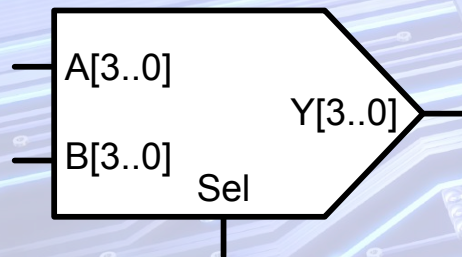
Else

Y <= B; -- de lo contrario (Sel = 1) Y = B

End if;

End process;

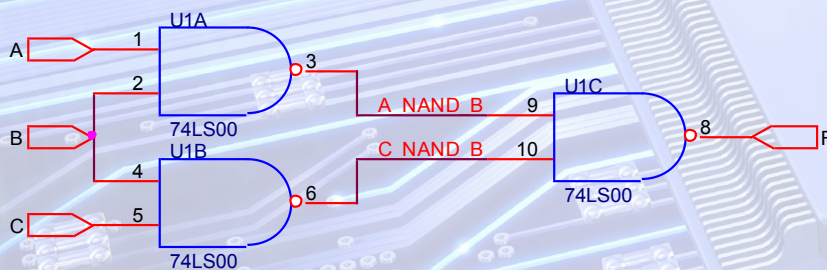
End Funcional;



8. Estilos de descripción

❖ Estilo Estructural

- Se describe la estructura del circuito, es decir los componentes que lo forman y como están interconectados entre sí
- Se suele usar para describir sistemas jerárquicos
- Generalmente se usa para describir el nivel superior (Diagrama de bloques)
- Descripción de un circuito combinacional a partir de puertas NAND de 2 entradas



-- Descripción del componente básico
-- Puerta Nand de 2 entradas

```
Library ieee;
Use ieee.std_logic_1164.all;
Entity Nand_2 is port(
  A, B : in std_logic;
  Y : out std_logic );
End Nand_2;
```

```
Architecture Funcional of Nand_2 is
Begin
  Y <= A Nand B;
End Funcional;
```

```
-- Descripción del circuito combinacional
-- a partir de las puertas Nand de 2 entradas
Library ieee;
Use ieee.std_logic_1164.all;
Entity Circuito_comb is
  port(A, B, C : in std_logic;
        Y : out std_logic );
End Circuito_comb;
```

Architecture Funcional of Circuito_comb is

```
  Component Nand_2 is port(
    A, B : in std_logic;
    Y : out std_logic );
```

```
End component Nand_2;
```

```
  signal A_NAND_B, C_NAND_B : std_logic;
```

Begin

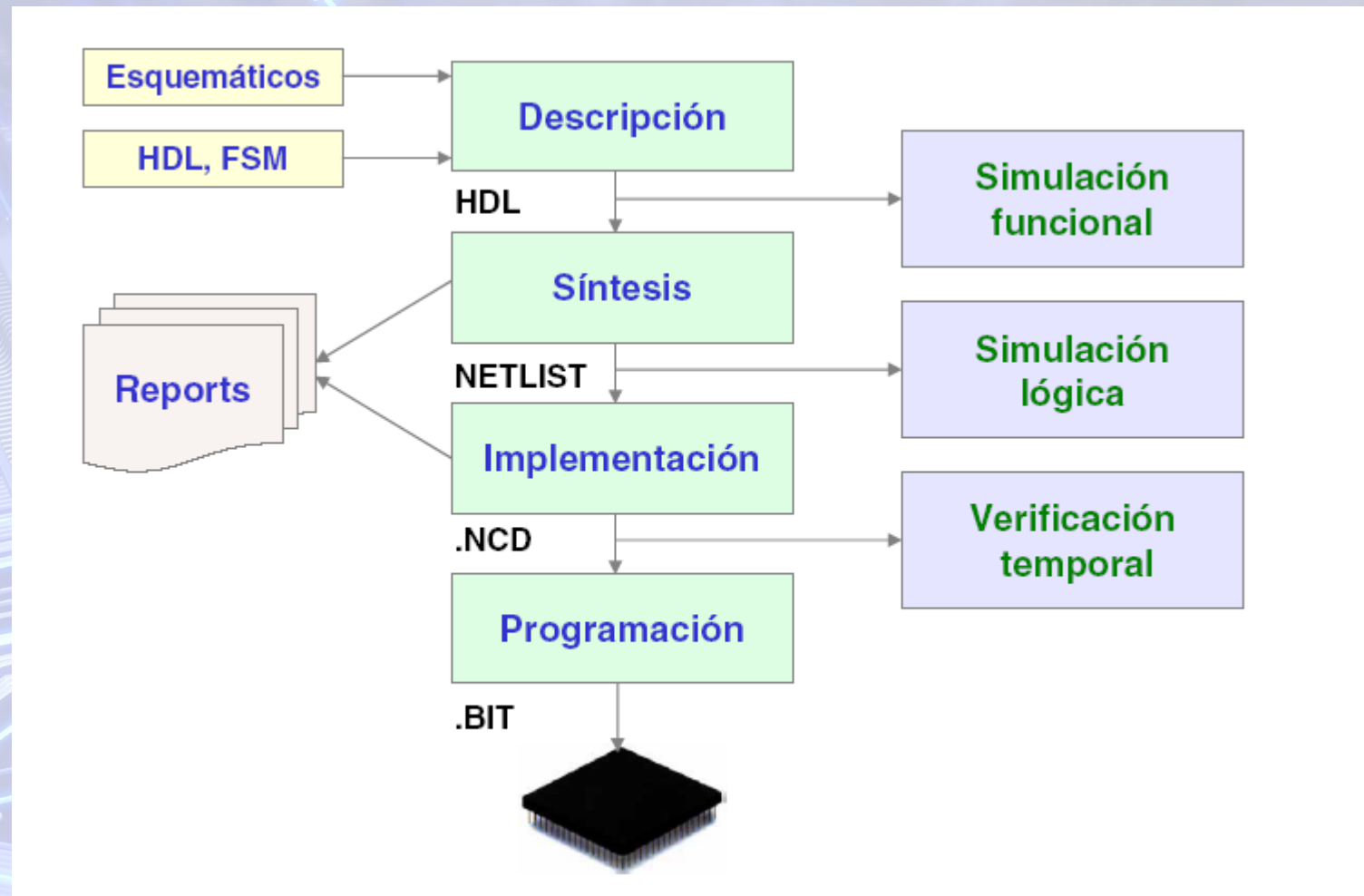
```
  U1A : Nand_2 port map (A, B, A_NAND_B);
```

```
  U1B : Nand_2 port map (B, C, C_NAND_B);
```

```
  U1C : Nand_2 port map (A_NAND_B, C_NAND_B, F);
```

End Funcional;

9. Flujo de diseño de FPGAs (Xilinx ISE)



8. Flujo de diseño de FPGAs (Xilinx ISE). Simulaciones VHDL con ISE Simulator

❖ Creando un proyecto en ISE

- File → New Project

New Project Wizard

← Create New Project
Specify project location and type.

Enter a name, locations, and comment for the project

Name:

Location: ...

Working Directory: ...

Description:

Select the type of top-level source for the project

Top-level source type:

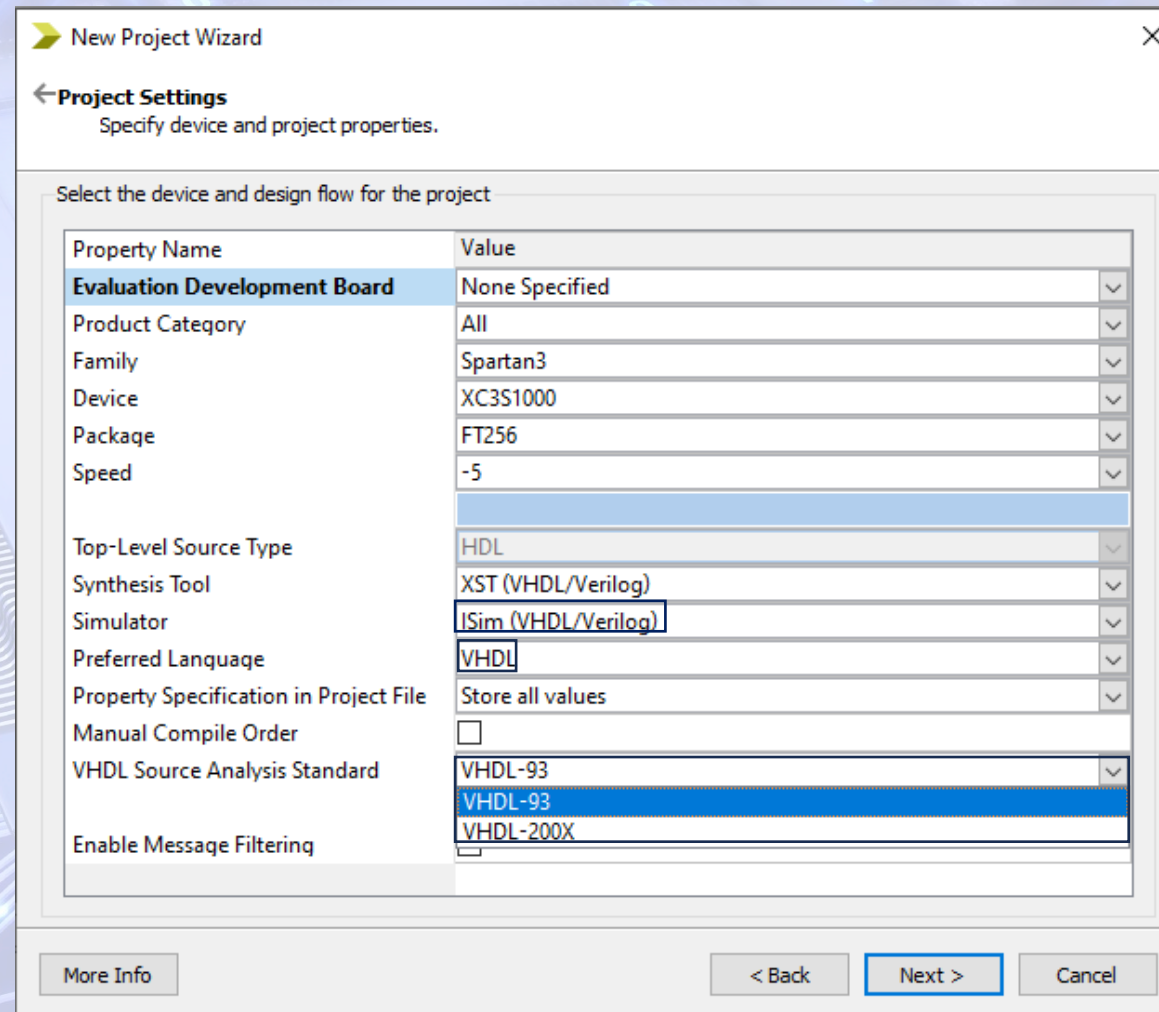
More Info Next > Cancel

Automáticamente se crea una carpeta con el nombre del proyecto

9. Flujo de diseño de FPGAs (Xilinx ISE). Simulaciones VHDL con ISE Simulator

❖ Creando un proyecto en ISE

- Seleccionamos como simulador **ISE Simulator (ISim)**



The screenshot shows the 'New Project Wizard' dialog box in Xilinx ISE, specifically the 'Project Settings' tab. The dialog is titled 'New Project Wizard' and has a close button (X) in the top right corner. Below the title bar, there is a section for 'Project Settings' with the instruction 'Specify device and project properties.' The main area of the dialog is titled 'Select the device and design flow for the project' and contains a table of properties.

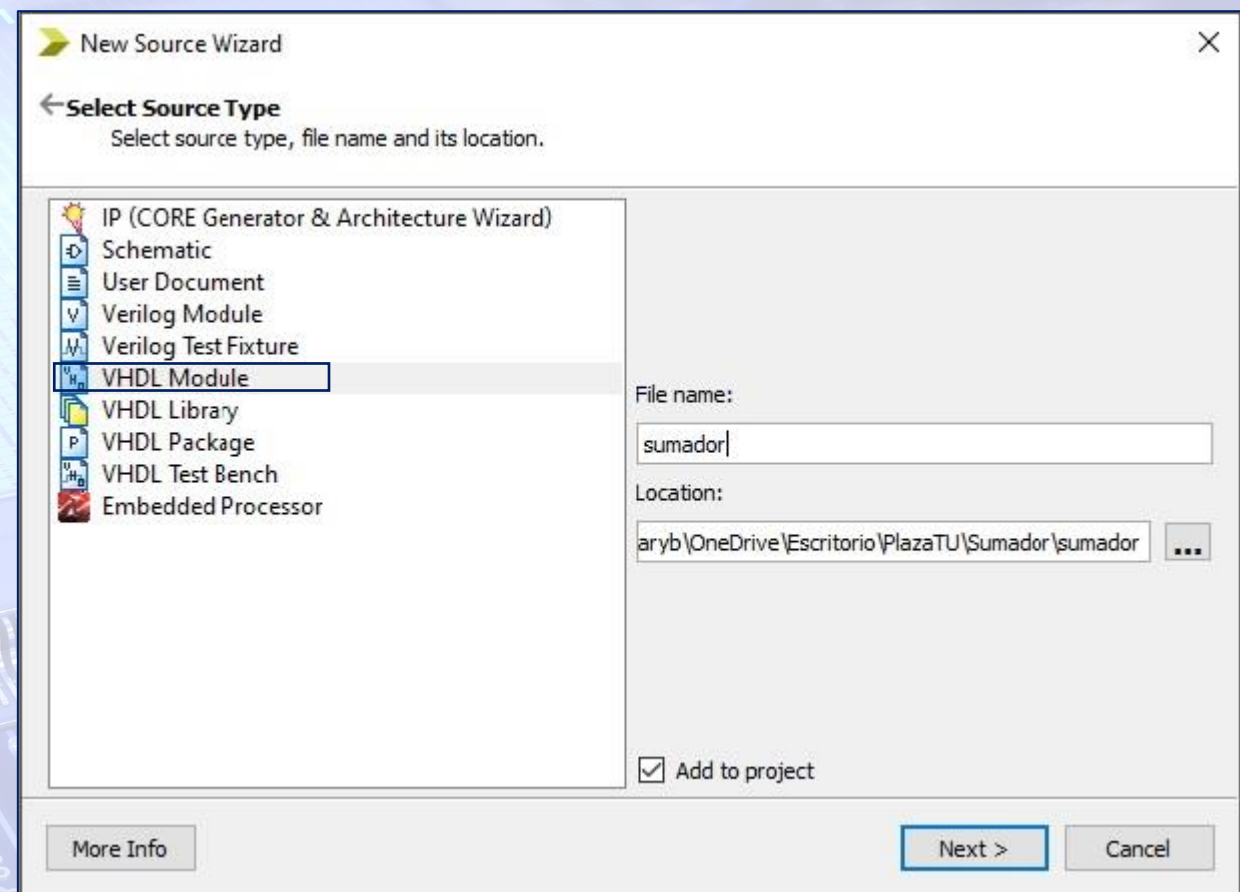
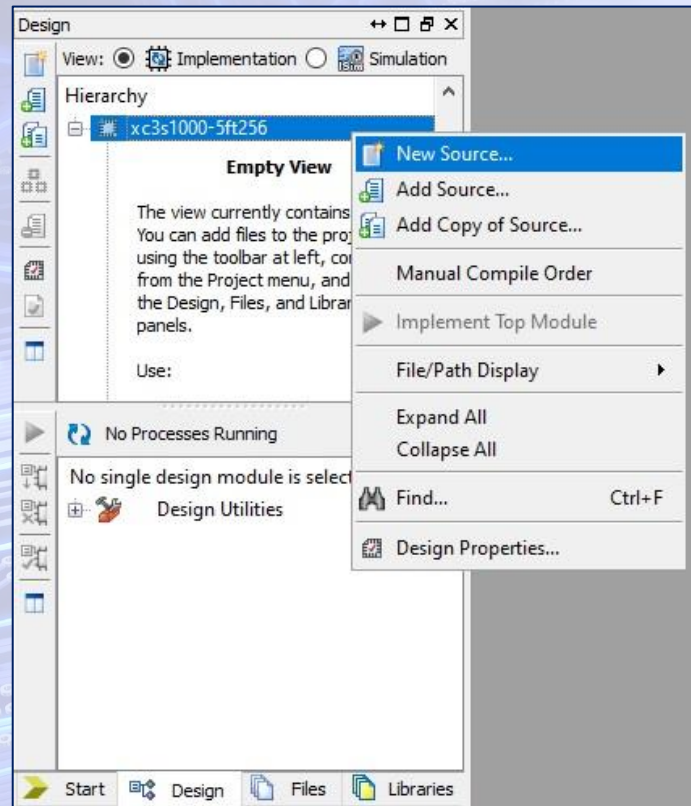
Property Name	Value
Evaluation Development Board	None Specified
Product Category	All
Family	Spartan3
Device	XC3S1000
Package	FT256
Speed	-5
Top-Level Source Type	HDL
Synthesis Tool	XST (VHDL/Verilog)
Simulator	ISim (VHDL/Verilog)
Preferred Language	VHDL
Property Specification in Project File	Store all values
Manual Compile Order	☐
VHDL Source Analysis Standard	VHDL-93
Enable Message Filtering	☐

At the bottom of the dialog, there are three buttons: 'More Info', '< Back', and 'Next >'. The 'Next >' button is highlighted with a blue border.

9. Flujo de diseño de FPGAs (Xilinx ISE). Simulaciones VHDL con ISE Simulator

❖ Creando un fichero VHDL

- Seleccionamos **New Source** y especificamos que es un **módulo VHDL**



9. Flujo de diseño de FPGAs (Xilinx ISE). Simulaciones VHDL con ISE Simulator

❖ Creando un fichero VHDL

- Especificar los puertos de la entidad y el nombre de la arquitectura

New Source Wizard

← Define Module
Specify ports for module.

Entity name:

Architecture name:

Port Name	Direction	Bus	MSB	LSB
A	in	☒	3	0
B	in	☒	3	0
LED	out	☒	6	0
	in	☐		
	in	☐		
	in	☐		
	in	☐		
	in	☐		
	in	☐		
	in	☐		

More Info < Back **Next >** Cancel

Al dar a siguiente sale una pantalla de resumen

New Source Wizard

← Summary
Project Navigator will create a new skeleton source with the following specifications.

Add to Project: Yes
Source Directory: C:\Users\maryb\OneDrive\Escritorio\PlazaTU\Sumador\sumador
Source Type: VHDL Module
Source Name: sumador.vhd

Entity name: sumador
Architecture name: Behavioral
Port Definitions:

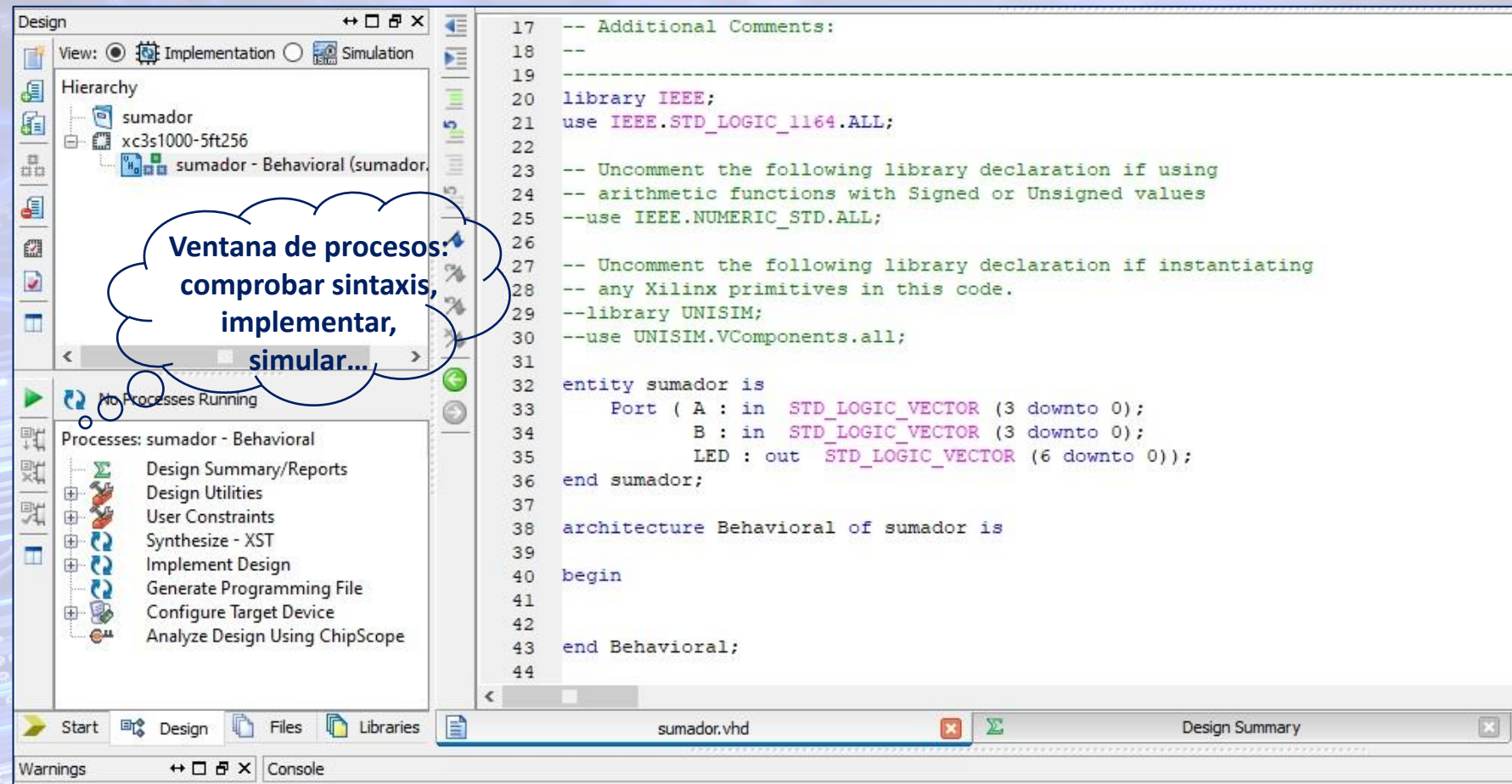
A	Bus:	3:0	in
B	Bus:	3:0	in
LED	Bus:	6:0	out

More Info < Back **Finish** Cancel



9. Flujo de diseño de FPGAs (Xilinx ISE). Simulaciones VHDL con ISE Simulator

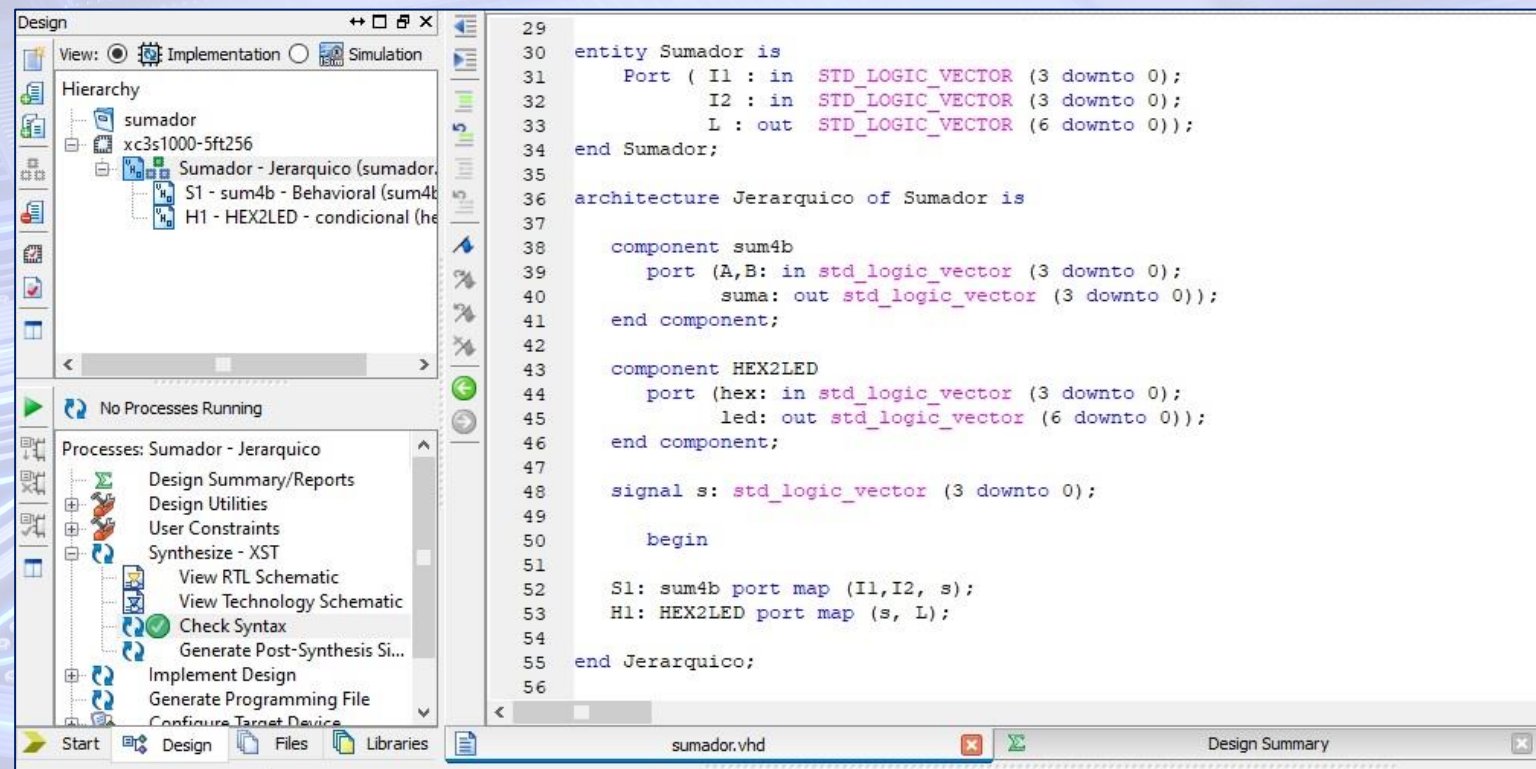
❖ Ventana de ISE



9. Flujo de diseño de FPGAs (Xilinx ISE). Simulaciones VHDL con ISE Simulator

❖ Terminando el código

- Antes de nada, tenemos que rellenar la arquitectura
- Después se comprueba que la sintaxis es correcta, pulsando en 'Check Syntax'



9. Flujo de diseño de FPGAs (Xilinx ISE). Simulaciones VHDL con ISE Simulator

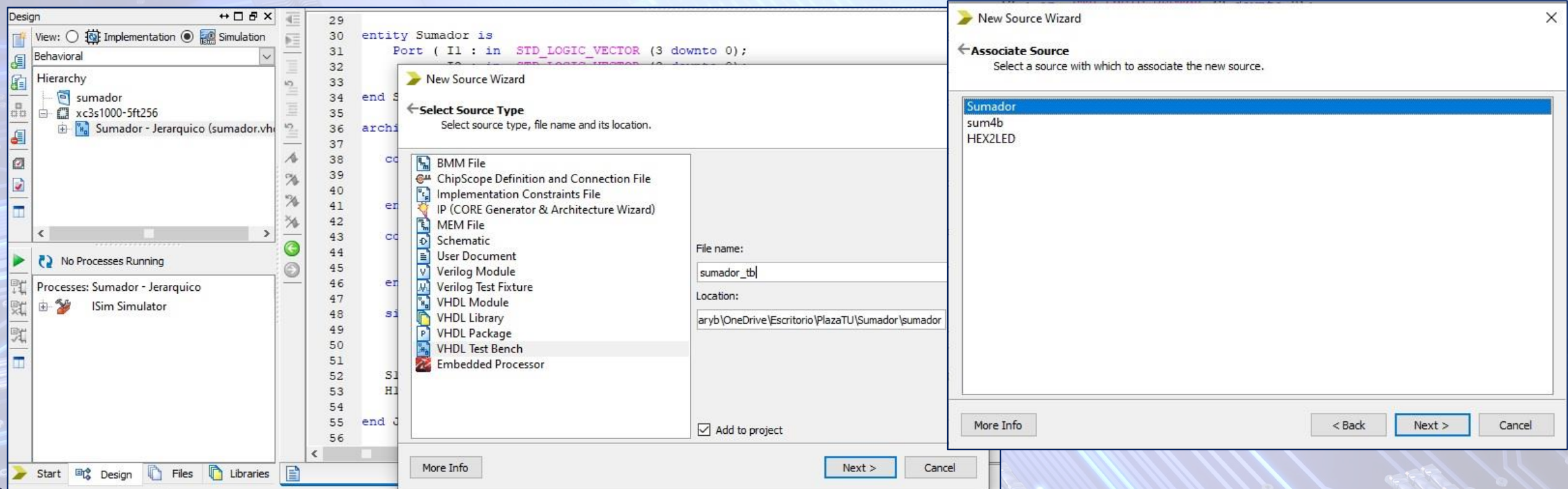
❖ Creación del fichero VHDL Testbench

- Antes de procesar el diseño con una herramienta de síntesis debe comprobarse su funcionamiento mediante una herramienta de simulación
 - ✓ Se denomina **simulación funcional**
- Una vez sintetizado el diseño es aconsejable simular el funcionamiento real del diseño, antes de programar el PLD
 - ✓ En este caso se hace una **simulación temporal**
- Tanto en la simulación funcional como en la temporal se debe describir los estímulos del circuito:
 - ✓ Indica como cambian las entradas respecto al tiempo
- VHDL permite crear un banco de pruebas (**test bench**) para describir los estímulos, y a su vez, también permite comprobar el funcionamiento de una forma automática
- Los test bench se describen en el nivel estructural de abstracción
- Se convierte en el nivel superior del diseño
- Se usan **process** para describir los estímulos y comprobar los valores de las señales de salida

9. Flujo de diseño de FPGAs (Xilinx ISE). Simulaciones VHDL con ISE Simulator

❖ Creación del fichero VHDL Testbench

- Señalar en vista la opción de **Simulation** para acceder al simulador de ISE
- Para crear un fichero testbench, hacer **Project → New Source** y seleccionar **VHDL Testbench**
- Seleccionar el módulo VHDL con el que se asociará el testbench



9. Flujo de diseño de FPGAs (Xilinx ISE). Simulaciones VHDL con ISE Simulator

❖ Estructura del fichero VHDL Testbench

```
31 ENTITY sumador_tb IS  
32 END sumador_tb;
```

Entidad vacía

```
33  
34 ARCHITECTURE behavior OF sumador_tb IS
```

```
35  
36 COMPONENT Sumador
```

```
37 PORT(  
38     I1 : IN  std_logic_vector(3 downto 0);  
39     I2 : IN  std_logic_vector(3 downto 0);  
40     L : OUT std_logic_vector(6 downto 0) );  
41 END COMPONENT;
```

Definición del
sistema a
simular como
componente

```
42  
43 signal I1 : std_logic_vector(3 downto 0) := (others => '0');  
44 signal I2 : std_logic_vector(3 downto 0) := (others => '0');  
45 signal L : std_logic_vector(6 downto 0);  
46
```

Definición de las señales que
contendrán los estímulos

```
47 BEGIN
```

```
48  
49 uut: Sumador PORT MAP (  
50     I1 => I1, I2 => I2, L => L);  
51
```

Instanciación del componente (asignación de estas señales a
los puertos de entrada/salida del componente)

```
52 stim_proc: process
```

```
53 begin
```

```
54     wait for 100 ns;
```

```
55
```

```
56     -- insert stimulus here
```

```
57
```

```
58     wait;
```

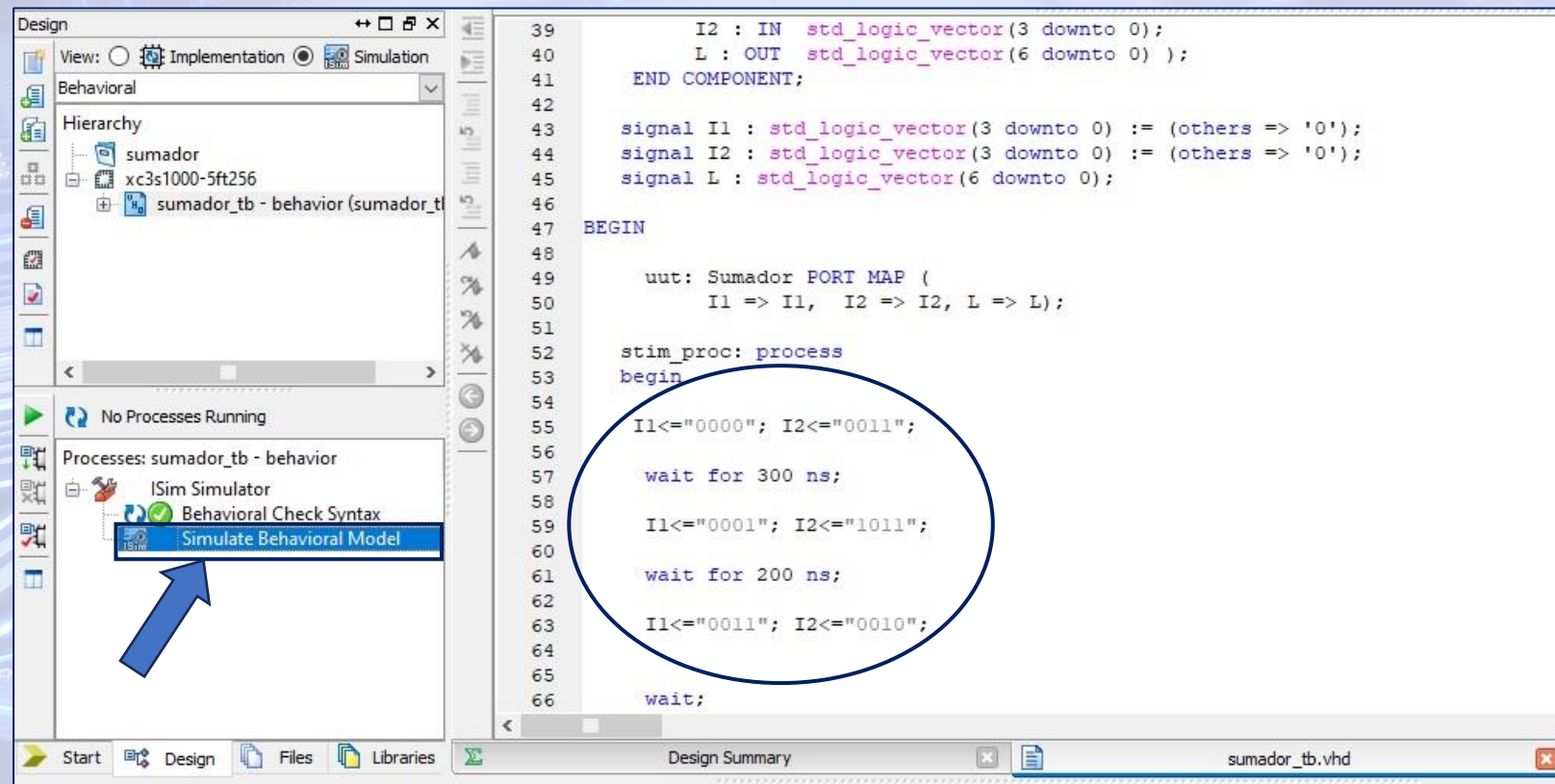
Uso de wait para finalizar el proceso

Definición de un proceso con
instrucciones del tipo wait para
introducir los valores de estímulos

9. Flujo de diseño de FPGAs (Xilinx ISE). Simulaciones VHDL con ISE Simulator

❖ Finalización del fichero VHDL Testbench

- Una vez finalizado el fichero de estímulos, se puede comprobar que la sintaxis del fichero es correcta y arrancar el simulador haciendo doble click sobre **Simulate Behavioral Model**



9. Flujo de diseño de FPGAs (Xilinx ISE). Simulaciones VHDL con ISE Simulator

❖ Corriendo la simulación

Zoom

Herramientas del simulador

Name	Value
i1[3:0]	3
i2[3:0]	2
l[6:0]	1101101

Se puede cambiar el formato en el que se visualizan las entradas y salidas



9. Flujo de diseño de FPGAs (Xilinx ISE). Síntesis

- ❖ Selección de la herramienta de síntesis

New Project Wizard

←Project Settings

Specify device and project properties.

Select the device and design flow for the project

Property Name	Value
Evaluation Development Board	None Specified
Product Category	All
Family	Spartan3
Device	XC3S1000
Package	FT256
Speed	-5
Top-Level Source Type	HDL
Synthesis Tool	XST (VHDL/Verilog)
Simulator	ISim (VHDL/Verilog)
Preferred Language	VHDL
Property Specification in Project File	Store all values
Manual Compile Order	☐
VHDL Source Analysis Standard	VHDL-93
Enable Message Filtering	☐

More Info

< Back

Next >

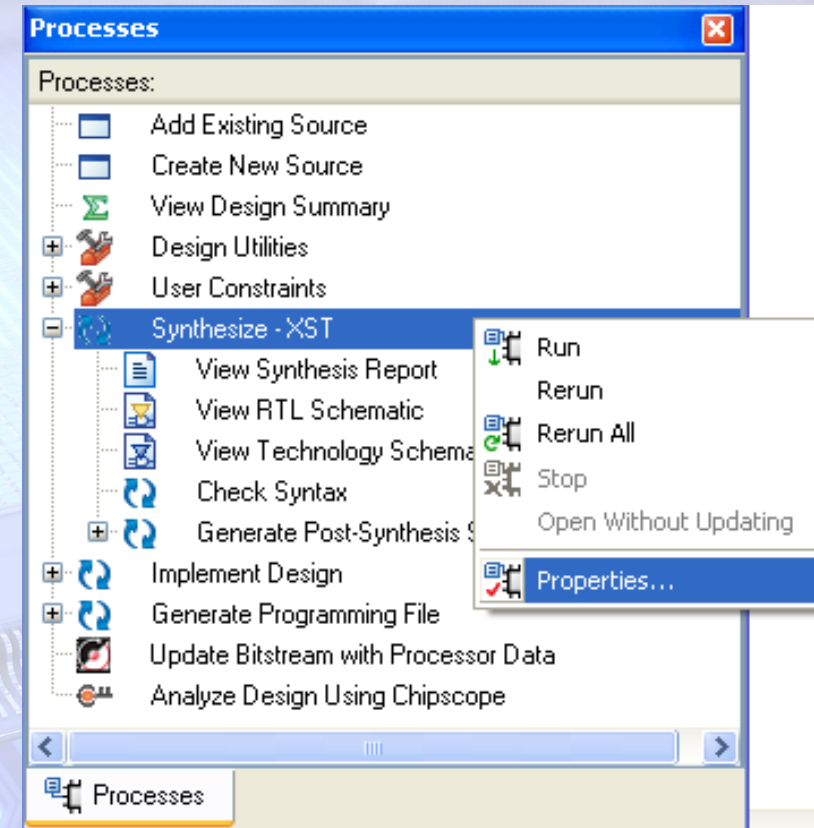
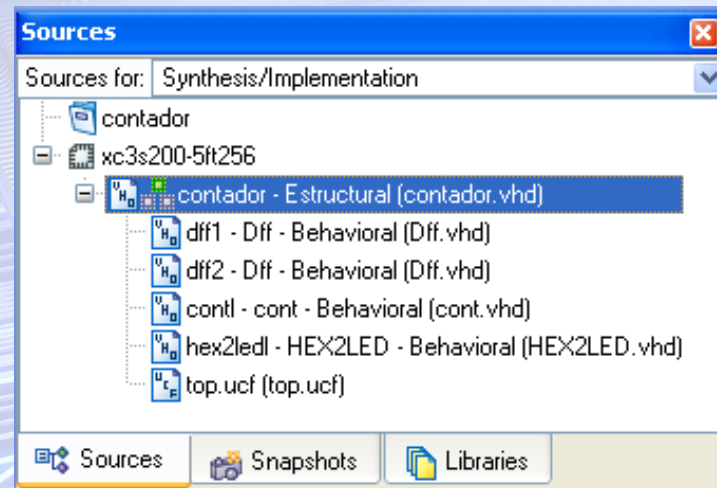
Cancel

Xilinx Synthesis Technology
(XST)

9. Flujo de diseño de FPGAs (Xilinx ISE). Síntesis

❖ Ejecución del proceso de síntesis

- Seleccionar el “top-level”
- Definir opciones de síntesis



9. Flujo de diseño de FPGAs (Xilinx ISE). Síntesis

❖ Opciones de síntesis

Process Properties

Category

- Synthesis Options
- HDL Options
- Xilinx Specific Options

Synthesis Options

Property Name	Value
Optimization Goal	Speed
Optimization Effort	Normal
Use Synthesis Constraints File	☒
Synthesis Constraints File	
Global Optimization Goal	AllClockNets
Generate RTL Schematic	Yes
Write Timing Constraints	☐
Verilog 2001	☒

Property display level: Standard

OK Cancel Apply

Process Properties

Category

- Synthesis Options
- HDL Options
- Xilinx Specific Options

HDL Options

Property Name	Value
FSM Encoding Algorithm	Auto
Safe Implementation	No
Case Implementation Style	None
RAM Extraction	☒
RAM Style	Auto
ROM Extraction	☒
ROM Style	Auto
Mux Extraction	Yes

Property display level: Standard

OK Cancel Apply

Process Properties

Category

- Synthesis Options
- HDL Options
- Xilinx Specific Options

Xilinx Specific Options

Property Name	Value
Add I/O Buffers	☒
Max Fanout	500
Register Duplication	☒
Equivalent Register Removal	☒
Register Balancing	No
Move First Flip-Flop Stage	☒
Move Last Flip-Flop Stage	☒
Pack I/O Registers into IOBs	Auto

Property display level: Standard Default

OK Cancel Apply Help



9. Flujo de diseño de FPGAs (Xilinx ISE). Síntesis

❖ Opciones de síntesis

Process Properties

Category

- Synthesis Options
- HDL Options
- Xilinx Specific Options

Synthesis Options

Property Name	Value
Optimization Goal	Speed
Optimization Effort	Normal
Use Synthesis Constraints File	☒
Synthesis Constraints File	
Global Optimization Goal	AllClockNets
Generate RTL Schematic	Yes
Write Timing Constraints	☐
Verilog 2001	☒

Property display level: Standard

OK Cancel Apply

Process Properties

Category

- Synthesis Options
- HDL Options
- Xilinx Specific Options

HDL Options

Property Name	Value
FSM Encoding Algorithm	Auto
Safe Implementation	No
Case Implementation Style	None
RAM Extraction	☒
RAM Style	Auto
ROM Extraction	☒
ROM Style	Auto
Mux Extraction	Yes

Property display level: Standard

OK Cancel Apply

Process Properties

Category

- Synthesis Options
- HDL Options
- Xilinx Specific Options

Xilinx Specific Options

Property Name	Value
Add I/O Buffers	☒
Max Fanout	500
Register Duplication	☒
Equivalent Register Removal	☒
Register Balancing	No
Move First Flip-Flop Stage	☒
Move Last Flip-Flop Stage	☒
Pack I/O Registers into IOBs	Auto

Property display level: Standard Default

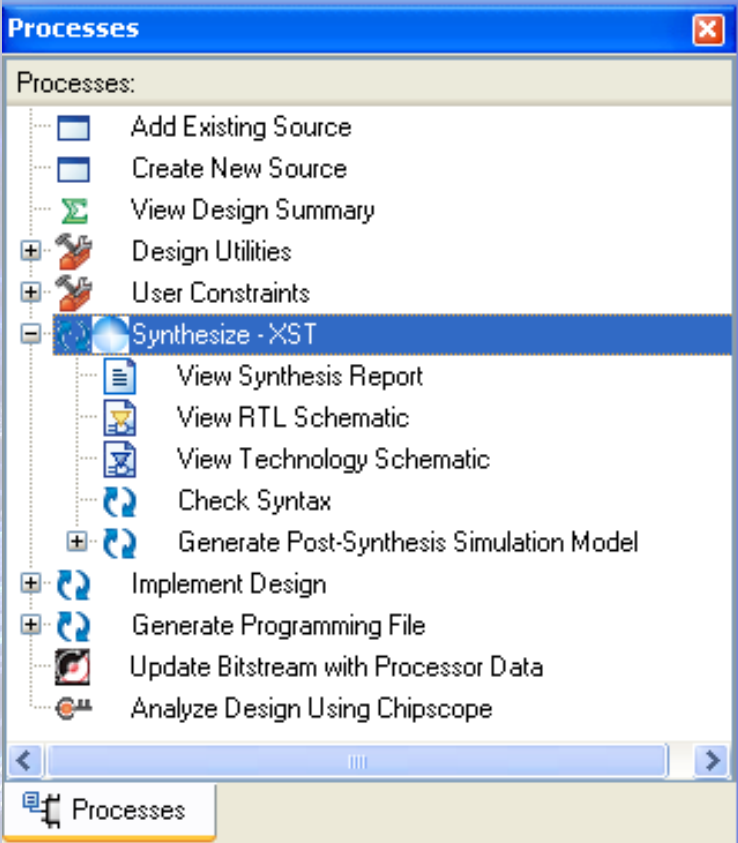
OK Cancel Apply Help



9. Flujo de diseño de FPGAs (Xilinx ISE). Síntesis

❖ Ejecución del proceso de síntesis

- Sintetizar y visualizar reports de síntesis



```
Release 8.2.03i - xst I.34
Copyright (c) 1995-2006 Xilinx, Inc. All rights reserved.
--> Parameter TMPDIR set to ./xst/projnav.tmp
CPU : 0.00 / 0.25 s | Elapsed : 0.00 / 0.00 s

--> Parameter xsthdmdir set to ./xst
CPU : 0.00 / 0.25 s | Elapsed : 0.00 / 0.00 s

--> Reading design: contador.prj
```

TABLE OF CONTENTS

- 1) Synthesis Options Summary
- 2) HDL Compilation
- 3) Design Hierarchy Analysis
- 4) HDL Analysis
- 5) HDL Synthesis
 - 5.1) HDL Synthesis Report
- 6) Advanced HDL Synthesis
 - 6.1) Advanced HDL Synthesis Report
- 7) Low Level Synthesis
- 8) Partition Report
- 9) Final Report
 - 9.1) Device utilization summary
 - 9.2) TIMING REPORT

```
=====
*                               Synthesis Options Summary                               *
=====
---- Source Parameters
Input File Name      : "contador.prj"
Input Format         : mixed
```

- Tabla de opciones de síntesis

- Mensajes generados durante la síntesis:
 - ✓ Lista de ficheros
 - ✓ Errores de sintaxis
 - ✓ Macros inferidos
 - ✓ Registros replicados

- Informe final:
 - ✓ Resumen de utilización
 - ✓ Informe de tiempos



9. Flujo de diseño de FPGAs (Xilinx ISE). Síntesis

❖ Reports de síntesis

Utilización

Opciones de síntesis

Mensajes de síntesis

Temporización

```
=====
*                               Synthesis Options Summary
=====
---- Source Parameters
Input File Name       : "contador.prj"
Input Format          : mixed
Ignore Synthesis Constraint File : NO

---- Target Parameters
Output File Name      : "contador"
Output Format         : NGC
Target Device         : xc3s200-5-ft256

---- Source Options
Top Module Name       : contador
Automatic FSM Extraction : YES
FSM Encoding Algorithm : Auto
FSM Style             : lut
RAM Extraction        : Yes
RAM Style             : Auto
ROM Extraction        : Yes
Mux Style             : Auto
Decoder Extraction    : YES
Priority Encoder Extraction : YES
Shift Register Extraction : YES
Logical Shifter Extraction : YES
XOR Collapsing       : YES
ROM Style            : Auto
Mux Extraction       : YES
Resource Sharing     : YES
Multiplier Style     : auto
```

```
=====
*                               HDL Synthesis
=====
Performing bidirectional port resolution...

Synthesizing Unit <dff>.
  Related source file is "C:/Archivos_C/EPS_cre60307/clases/DAEO/Lab2/cont
  Found 1-bit register for signal <Q>.
  Summary:
    inferred 1 D-type flip-flop(s).
Unit <dff> synthesized.

Synthesizing Unit <cont>.
  Related source file is "C:/Archivos_C/EPS_cre60307/clases/DAEO/Lab2/cont
  Found 4-bit updown counter for signal <count>.
  Summary:
    inferred 1 Counter(s).
Unit <cont> synthesized.

Synthesizing Unit <hex2led>.
  Related source file is "C:/Archivos_C/EPS_cre60307/clases/DAEO/Lab2/cont
```

```
=====
*                               Estadística general
=====
HDL Synthesis Report

Macro Statistics
# ROMs                : 1
16x7-bit ROM          : 1
# Counters            : 1
4-bit updown counter  : 1
# Registers           : 2
1-bit register        : 2

=====
*                               Advanced HDL Synthesis
=====
Loading device for application Rf_Device from file '3s200.nph' in environme
nt.

=====
Advanced HDL Synthesis Report

Macro Statistics
# ROMs                : 1
16x7-bit ROM          : 1
# Counters            : 1
4-bit updown counter  : 1
# Registers           : 1
Flip-Flops            : 1
```

Device utilization summary:

Selected Device : 3s200ft256-5

Number of Slices:	7	out of	1920	0%
Number of Slice Flip Flops:	6	out of	3840	0%
Number of 4 input LUTs:	14	out of	3840	0%
Number of IOs:	19			
Number of bonded IOBs:	19	out of	173	10%
Number of GCLKs:	1	out of	8	12%

Clock Signal	Clock buffer (FF name)	Load
reloj_global	BUFGP	2
c(cl:O)	NONE (*) (cont1/count_1)	4

(*) This 1 clock signal(s) are generated by combinatorial logic, and XST is not able to identify which are the primary clock signals. Please use the CLOCK_SIGNAL constraint to specify the clock signal(s) generated by combinatorial logic. INFO:Xst:2169 - HDL ADVISOR - Some clock signals were not automatically buffered by XST with BUFG/BUFR res

Asynchronous Control Signals Information:

Control Signal	Buffer (FF name)	Load
Reset	IBUF	6

Timing Summary:

Speed Grade: -5

Minimum period: 2.912ns (Maximum Frequency: 343.389MHz)
Minimum input arrival time before clock: 3.538ns
Maximum output required time after clock: 7.962ns
Maximum combinational path delay: 6.407ns

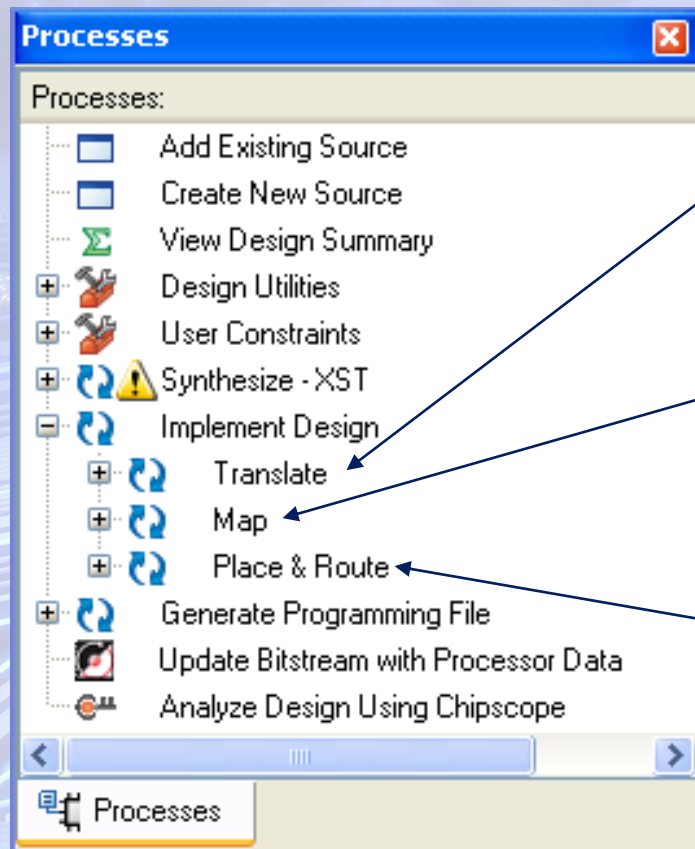
Timing Detail:

All values displayed in nanoseconds (ns)



9. Flujo de diseño de FPGAs (Xilinx ISE). Implementación

❖ Etapa de implementación: Fases



Combina múltiples ficheros de descripciones en un solo netlist- **ngdbuild**

Asocia símbolos lógicos del netlist a componentes físicos de la FPGA- **map**

Colocar los componentes en la FPGA, los conecta y extrae información temporal- **par**

9. Flujo de diseño de FPGAs (Xilinx ISE). Implementación

❖ Ejecución del proceso de implementación

Opciones de implementación

Seleccionar el "top-level"

Sources

Sources for: Synthesis/Implementation

contador

xc3s200-5ft256

contador - Estructural (contador.vhd)

dff1 - Dff - Behavioral (Dff.vhd)

dff2 - Dff - Behavioral (Dff.vhd)

cont1 - cont - Behavioral (cont.vhd)

hex2led1 - HEX2LED - Behavioral (HEX2LED.vhd)

top.ucf (top.ucf)

Sources

Snapshots

Libraries

Ejecutar el proceso

Processes

Processes:

Implement Design

Translate

Translation Report

Floorplan Design

Generate Post-Translate Simulation Model

Assign Package Pins Post-Translate

Map

Map Report

Generate Post-Map Static Timing

Floorplan Design Post-Map (Floorplanner)

Manually Place & Route (FPGA Editor)

Generate Post-Map Simulation Model

Place & Route

Place & Route Report

Clock Region Report

Asynchronous Delay Report

Pad Report

Guide Results Report

MPPR Results Utilities

Generate Post-Place & Route Static Timing

View/Edit Placed Design (Floorplanner)

View/Edit Routed Design (FPGA Editor)

Analyze Power (XPower)

Generate Power Data

Generate Post-Place & Route Simulation Model

Generate IBIS Model

Back-annotate Pin Locations

Generate Programming File

Processes

Process Properties

Category

Translate Properties

Map Properties

Place & Route Properties

Property Name

Value

Macro Search Path

Allow Unmatched LOC Constraints

Trim Unconnected Signals

Generate Detailed MAP Report

MAP Guide Design File (.ncd)

MAP Guide Mode

Pack I/O Registers/Latches into IOBs

Place And Route Mode

Place & Route Effort Level (Overall)

Starting Placer Cost Table (1-100)

PAR Guide Design File (.ncd)

PAR Guide Mode

Use Timing Constraints

Use Bonded I/Os

Generate Asynchronous Delay Report

Generate Post-Place & Route Static Timing Report

Generate Post-Place & Route Simulation Model

Number of PAR Iterations (0-100)

Number of Results to Save (0-100)

Save Results in Directory (.dir will be appended)

Power Reduction

Property display level: Standard

Default

OK

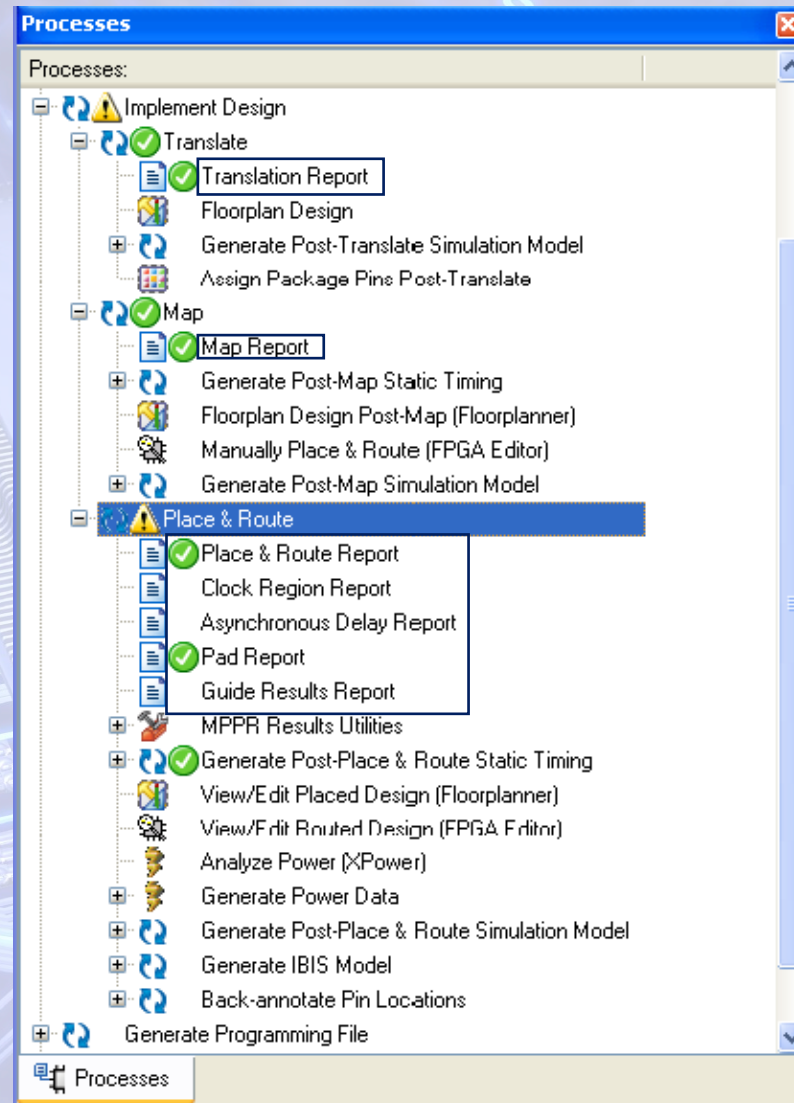
Cancel

Apply

Help

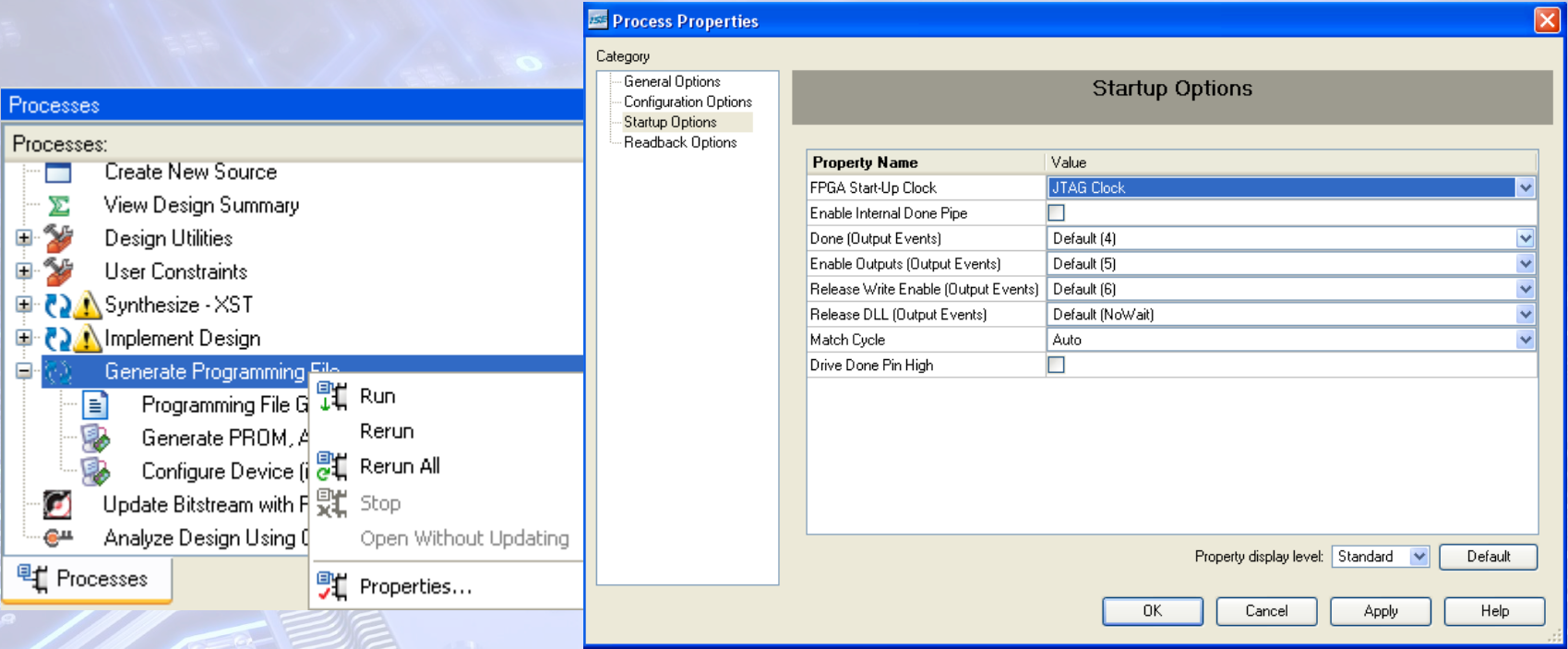
9. Flujo de diseño de FPGAs (Xilinx ISE). Implementación

❖ Reports de implementación

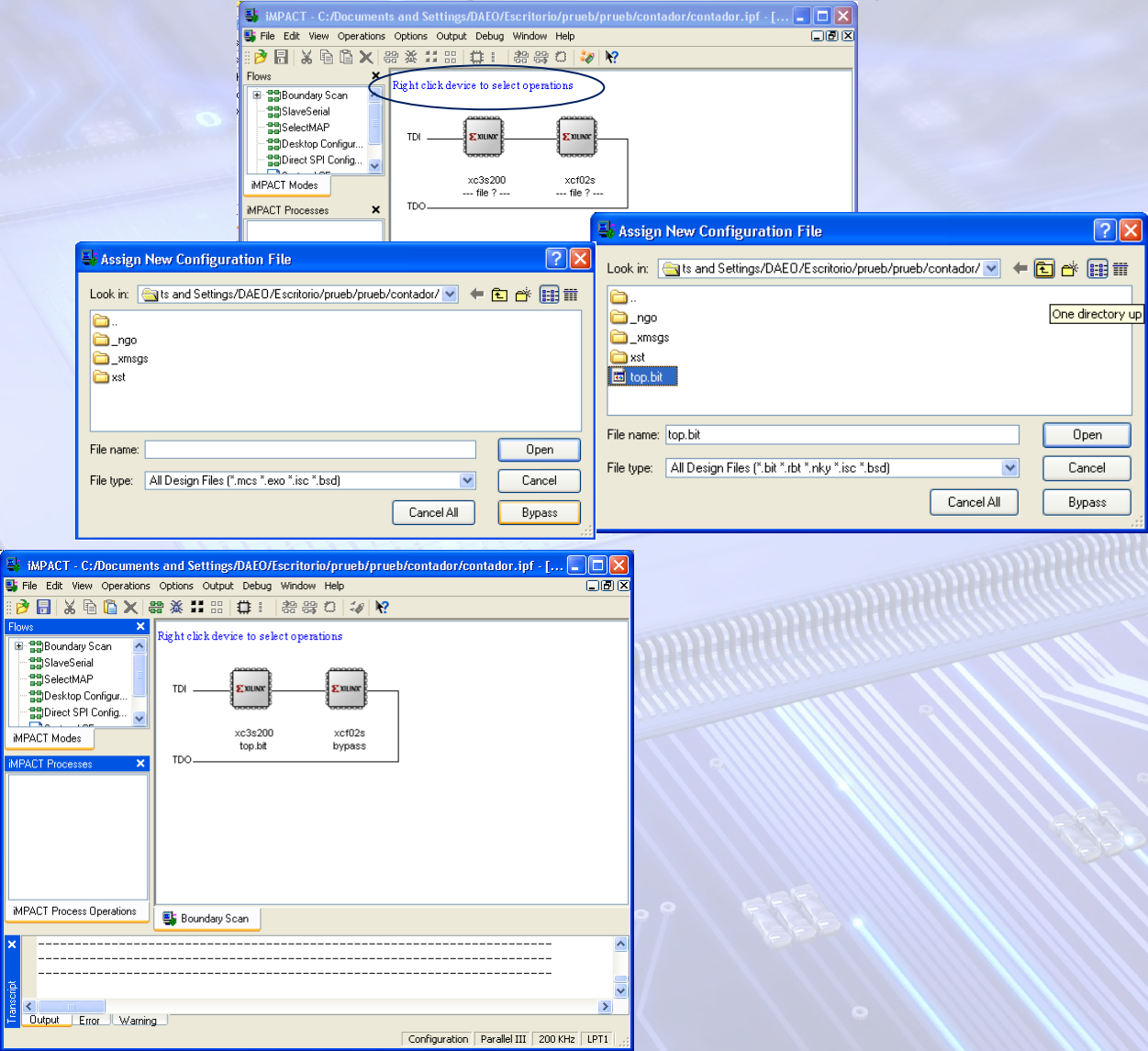


- Son generados de forma automática en las distintas fases de implementación
- Contienen información sobre el diseño:
 - Avisos y errores del software
 - Opciones utilizadas
 - Estadísticas:
 - ✓ Utilización de dispositivos
 - ✓ Prestaciones frente a restricciones
 - ✓ Relaciones de terminales E/S (Pinout)

9. Flujo de diseño de FPGAs (Xilinx ISE). Programación de la FPGA

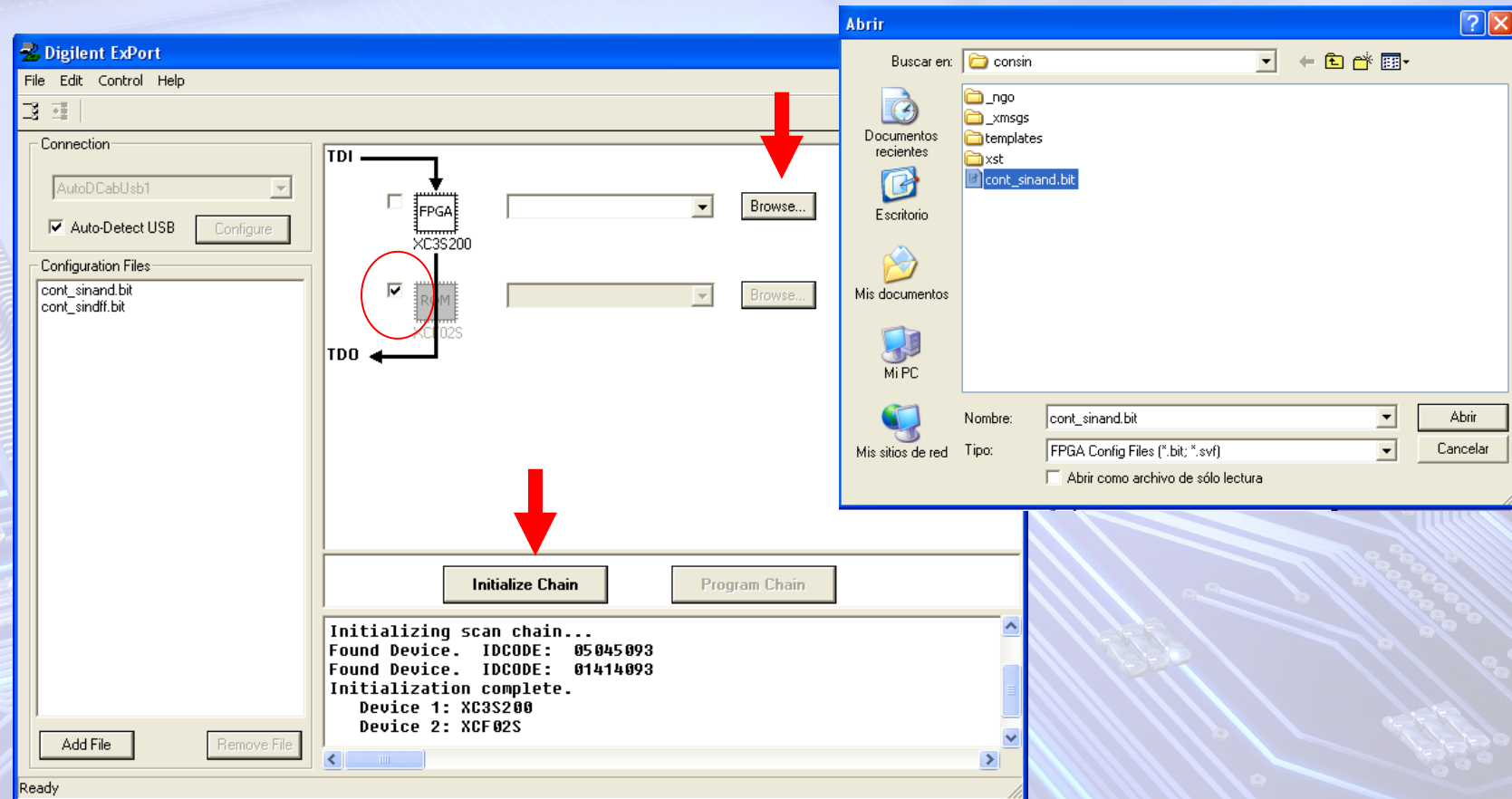


9. Flujo de diseño de FPGAs (Xilinx ISE). Programación de la FPGA. Configure Device (Impact)



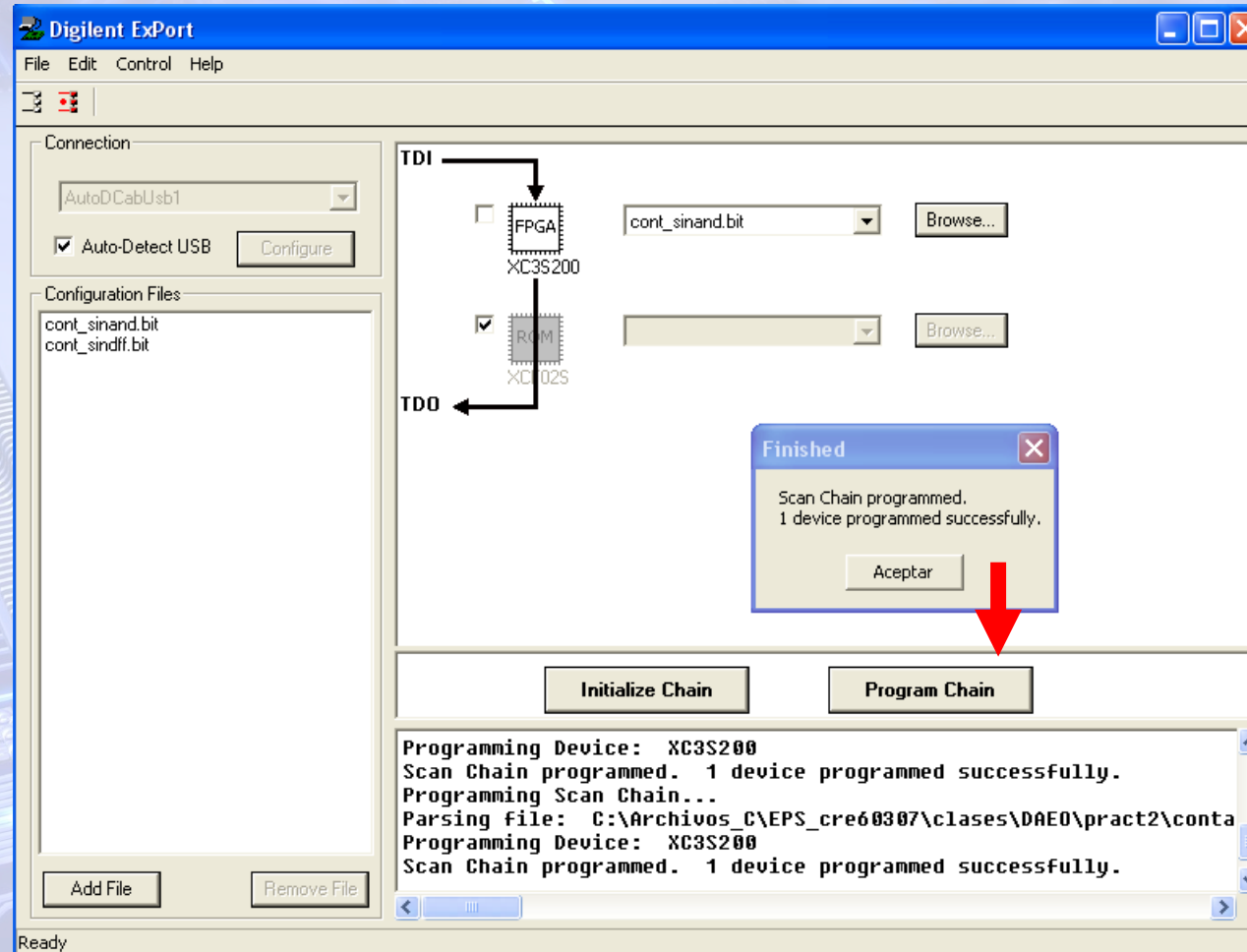
9. Flujo de diseño de FPGAs (Xilinx ISE). Programación de la FPGA. Export (JTAG-USB)

❖ Inicio -> Programas -> Digilent -> Adept -> Export



9. Flujo de diseño de FPGAs (Xilinx ISE). Programación de la FPGA. Export (JTAG-USB). Program Chain

➔ Visualizar operación en la FPGA



10. Problemas propuestos

1. Encontrar los errores de sintaxis en el siguiente fichero fuente

```
entity muchos_erroses is port                                --línea 1
    a: bit_vector(3 to 0);                                   --línea 2
    b: out std_logic_vector(0 to 3);                         --línea 3
    c: in bit_vector(5 downto 0);                           --línea 4
end muchos_erroses                                          --línea 5
architecture no_correcta of muchos_erroses                 --línea 6
    signal reg std_logic_vector(7 downto 0);                --línea 7
begin                                                       --línea 8
etiqueta: process                                           --línea 9
    begin                                                    --línea 10
        if c = x"F" then                                    --línea 11
            b <= a                                           --línea 12
        else                                                 --línea 13
            b <= '0101';                                     --línea 14
        end if                                              --línea 15
    end process;                                           --línea 16
    reg := reg sll 2;                                       --línea 17
end no_correcta                                           --línea 18
```


10. Problemas propuestos

2. Escribir la declaración de la entity para la siguiente architecture, considerando que todas las signals de la architecture son ports.

```
architecture escribe_entity of problema is
begin
  decod_memo: process(addr)
  begin
    shadow_ram_sel <= '0';
    sram_sel <= '0';
    if addr >= x"0100" and addr < x"4000" then
      shadow_ram_sel <= '1';
    elsif addr >= x"8000" and addr < x"C000" then
      sram_sel <= '1';
    end if;
    promsel <= '0';
    if mem_mapped = '0' and bootup = '1' then
      prom_sel <= '1';
    end if;
  end process decod_memo;
  mem_mapped <= shadow_ram_sel or sram_sel;
end escribe_entity;
```


10. Problemas propuestos

3. ¿Cuál es la ventaja de usar la asignación siguiente para el array a?

```
architecture problema of asignacion
is
  signal a : std_logic_vector(31 downto 0);
begin
  a <= (others => '1');
  ...
end problema;
```


10. Problemas propuestos

4. Determinar el valor de cada bit de los arrays del siguiente segmento de código VHDL.

```
Variable A: std_logic_vector(0 to 3) := (3 => '1', Others => '0');  
Variable B: std_logic_vector(6 downto 3);  
Variable C: std_logic_vector(7 downto 0) := X"5A";  
Variable D: std_logic_vector(0 to 7);
```

```
B := "0110";  
A := B;  
-- se usa el valor actualizado de A  
C(5 downto 0) := A(0 to 1) & B;  
D := B & C(7 downto 4) ;  
A := '1' & B(6) & "01";
```

10. Problemas propuestos

5. Declarar la matriz de celdas de una ROM de organización 16 x 8 con el contenido de la siguiente tabla.

Dirección	Contenido	Dirección	Contenido
0	0	8	87 ₁₆
1	3 ₁₆	9	44 ₁₆
2	45 ₁₆	A	B9 ₁₆
3	EF ₁₆	B	AA ₁₆
4	55 ₁₆	C	FF ₁₆
5	2C ₁₆	D	58 ₁₆
6	79 ₁₆	E	9D ₁₆
7	AD ₁₆	F	66 ₁₆

10. Problemas propuestos

6. Dada la declaración de las siguientes señales, calcúlese el resultado de aplicar los operadores lógicos al contenido inicial de dichas señales

```
Signal oper_A : std_logic_vector(7 downto 0) := "10011010";
```

```
Signal oper_B : std_logic_vector(7 downto 0) := "11010110";
```

```
Signal Resul : std_logic_vector(7 downto 0) ;
```

```
-----  
Resul <= not oper_A;
```

```
Resul <= oper_A and oper_B;
```

```
Resul <= oper_A xor oper_B;
```

```
Resul <= oper_A nor oper_B;
```


10. Problemas propuestos

7. Dada la declaración de la siguiente señal, calcúlese el resultado de aplicar los operadores de desplazamiento al contenido inicial de dicha señal

Signal registro : bit_vector(7 **downto** 0) := "10011010";

registro <= registro **sll** 2;
registro <= registro **srl** 4;
registro <= registro **rol** 4;
registro <= registro **ror** 5;

10. Problemas propuestos

8. Descríbase en VHDL el circuito combinacional, cuya salida viene dada por la siguiente tabla de verdad. Se usará en la arquitectura un statement de asignación al puerto de salida F, empleando los operadores lógicos correspondientes.

a	b	c	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0